VMA — Visual Macro Assembler

Nyelvi kézikönyv · 2.2.0-s verzió

A VMA a C64 Visual Assembler 6502 / 6510 assemblere — az Expert mód nyelve, és az a szöveg, amiként a blokkprogramok leíródnak. Ez a kézikönyv a nyelvet írja le; a szerkesztő, a blokk-paletta és az app többi eszköze a C64 Visual Assembler kézikönyvében található.

Tartalom

1. Forrásformátum
2. Számok és kifejezések
3. Kimenet: PRG, BASIC stub és belépési pont
4. Címzési módok
5. 6502 utasítások
6. Direktívák és makrók — referencia
7. KickAssembler-stílusú szintaxis
8. Parancssor
9. Ismert korlátok

---

1. Forrásformátum

A VMA forrás sima szöveg, soronként egy utasítás. A mnemonikoknál és a direktíváknál a kis- és nagybetű nem számít (`lda` , `LDA` , `.BYTE` , `.byte`); a címkenevek kis-/nagybetű-érzékenyek.

```
; megjegyzés — a // is megjegyzés
* = $1000                ; ORG: fordítás $1000-tól
start:                   ; címke (a kettőspont kötelező)
    lda #$06             ; utasítás: mnemonik és operandus
    sta $D020
    .byte $01, $02, $FF  ; a direktíva ponttal kezdődik
loop: dex                ; címke és utasítás egy sorban
    bne loop
    rts
```

- **Megjegyzés:** `;` vagy `//` után a sor végéig (idézőjeles szövegen belül nem).
- **Címke:** `név:` — betű, számjegy és `_` , nem kezdődhet számjeggyel. Állhat külön sorban vagy utasítás előtt. A ponttal kezdődő név (`.loop:`) **lokális címke** (lásd Lokális (pontos) címkék); az önálló `-` és `+` **anonim címke**, hivatkozás rá `bne -` / `beq +` (a legközelebbi előtte / utána).
- **Utasítás:** mnemonik és operandus; a behúzás szabad.
- **Direktíva:** ponttal kezdődik (`.byte` , `.macro` , `.if` …); a `* = $xxxx` (vagy `.pc = $xxxx`) a címet állítja.
- **Saját makró hívása:** `:név(args)` , `.invoke név(args)` , `.call név(args)` vagy `név(args)` .

2. Számok és kifejezések

Forma	Példa	Érték
Hexadecimális	`$D020`, `$0F`	`$` előtag
Bináris	`%00001111`	`%` előtag
Decimális	`255`, `40*25`	előtag nélkül
Karakter	`'A'`	a kódja (`$41`)
Aktuális cím	`*`	a sor címe (lásd A `*` program counter)

Az utasítások operandusában a hexa számot `$`-ral írd. A könyvtári makrók cím-argumentumaiban (`.rawbytes C000, …`, `.sprite_init 0, 1, 80`) az egy-négy jegyű puszta szám hexának számít.

Műveletek (operandusban, `.const`-ban, `.fill`-ben, `.assert`-ben, `.if`-ben): `+ - * / %`, `& | ^ ~`, `<< >>`, zárójelek, `<kif` (alsó bájt) és `>kif` (felső bájt), feltételekben `== != < > <= >=` (igaz = 1, hamis = 0). A `.fill`, a `.const` és társaik ismerik a `lo()`, `hi()`, `sin()`, `cos()`, `round()`, `min()`, `max()`, `abs()` függvényt és a `PI`-t is.

```
.const B = 7
    lda #(B+1)*2
    lda #B & 3 | 8
    lda #~B & $FF
    ldx #'A'
    lda #<screen        ; egy címke alsó bájtja
    lda #>(*+63)        ; egy PC-relatív cím felső bájtja
```

3. Kimenet: PRG, BASIC stub és belépési pont

A VMA C64 **PRG**-t ír: kétbájtos betöltési cím, utána a memóriakép. Minden ORG-szakasz és minden saját címre adatot tevő direktíva (`.rawbytes`, `.incbin`, `.sid` …) egy képbe kerül; a köztük lévő rések nullák.

- **ORG nélkül** a program a `$0801`-en, a BASIC elején kezdődik.
- **BASIC SYS stub** (alapértelmezés): a `$0801`-re egy `10 SYS nnnn` sor kerül, így a program `RUN`-nal indul. A `$0801`-es első ORG-ú program ilyenkor `$080D`-n kezdődik. A stub a `--no-basic-stub` kapcsolóval kapcsolható ki; nélküle a `$0801`-es program `$C000`-ra fordul, minden más az első ORG-ján marad.
- **Belépési pont:** a SYS sor a kód elejére ugrik, vagy az első sor `; Entry point: $1234` megjegyzésében megadott címre.
- **A BASIC sor alá eső kód** (`$0801` alatti ORG a stubbal együtt): egy kis betöltő indítja és teszi a helyére a programot, így az továbbra is `RUN`-nal indul.
- A saját BASIC SYS sort író forrás (`.byte $0B, $08, $0A, $00, $9E, …` a `$0801`-en) változatlanul kerül a PRG-be, második stub nélkül. A sor a forrásban bárhol állhat, a kód után és címke mögött is (ahogy a Restore64 visszafejtései írják); a `$0801`-en marad (2.1.1-től).

A PRG-ből a parancssor **cartridge**-et és **lemezképet** is készít (2.1.0-tól, lásd 8. Parancssor):

- `--crt FÁJL` — Magic Desk cartridge (CRT 19-es típus, 8 × 8K bank, legfeljebb 65 408 bájt program). A 0. bank loadere inicializálja a KERNAL-t, majd a RAM-ban, a `$0100–$013B` területen futó másoló rutin a betöltési címére

másolja a programot (a `$DE00`-n keresztül bankot váltva), kikapcsolja a cartridge-et és a belépési pontra ugrik. A program ne essen a `$0100–$013B` területre, se a loader által használt `$02–$04` / `$FB–$FE` zero page bájtokra.
- `--d64 [FÁJL]` — 1541-es lemezkép (35 sáv) a PRG-vel a saját nevén, és a `--add` fájlokkal, úgy elrendezve, ahogy az 1541 DOS írja; c1541 nem kell.

---

4. Címzési módok

Minden 6502 utasítás egy vagy több címzési módot támogat. A VMA az operandus írásmódjából választja ki a módot.

Mód	Címke	Példa	Leírás
implied	Implied	`NOP`	Nincs operandus; az utasítás önmagában teljes
immediate	Immediate	`LDA #$FF`	Inline konstans; az assembler automatikusan hozzáadja a `#`-t
zeroPage	Zero page	`LDA $10`	Egy byte-os cím a nulladik lapon (0–255)
zeroPageX	Zero page,X	`LDA $10,X`	Zero page cím + X regiszter offset (az eredmény a 0. lapon körbefordul)
zeroPageY	Zero page,Y	`LDX $FB,Y`	Zero page cím + Y regiszter offset
absolute	Absolute	`LDA $0400`	Teljes 16 bites memóriacím
absoluteX	Absolute,X	`LDA $0400,X`	16 bites cím + X regiszter offset
absoluteY	Absolute,Y	`LDA $0400,Y`	16 bites cím + Y regiszter offset
relative	Relative/Label	`BNE loop`	Elágazás utasításokhoz; adj meg egy címkenevet vagy célcímet
indirectX	Indirect,X	`LDA ($FB,X)`	Zero page indexelt indirekt (operandus = zero page cím, 1 byte)
indirectY	Indirect,Y	`LDA ($FB),Y`	Zero page indirekt indexelt (operandus = zero page cím, 1 byte)
indirect	Indirect	`JMP ($0100)`	Indirekt; csak JMP-vel használható

Konstansok a zero page-en

A csak konstansokból (`.const`, `név = érték`, `.var`) álló, egy bájtba férő operandus az utasítás **zero page** módját kapja, pontosan úgy, mint maga a szám: `.const zp = $FB` után az `lda zp` `A5 FB` lesz (2 bájt, 3 ciklus), az `lda zp,x` pedig `B5 FB`. Ha az utasításnak nincs zero page változata az adott módban, abszolút marad (`lda zp,y` → `B9 FB 00`). Címkére nem szűkít — a címe csak az elrendezés után ismert —, az `lda.abs zp` pedig kikényszeríti az abszolút módot. A szám megtartja a leírt szélességét: a `$FB` zero page, a `$00FB` abszolút. *(Új a 2.1-ben.)*

Címke-kifejezések operandusként

Bármely operandus, amely címet vagy immediate értéket elfogad, közvetlenül elfogad egy **konstans nevet** is (`.const`-ot vagy címkét). Ezen felül **címke+offset** vagy **címke−offset** kifejezésekkel hivatkozhatsz egy nevesített konstanshoz képesti címre:

Szintaxis	Példa	Leírás
`label`	`STA screen_ram,X`	A címke/konstans értékére oldódik
`label+$hex`	`STA screen_ram+$0100,X`	Címke-cím plusz hex offset

Szintaxis	Példa	Leírás
`label+decimal`	`STA screen_ram+256,X`	Címke-cím plusz decimális offset
`label-$hex`	`LDA table-$10`	Címke-cím mínusz hex offset
`$hexa+kifejezés`	`LDA $1234+1`, `STA $0400+40*2,X`	Számokból, címkékből és konstansokból álló bármely kifejezés *(2.1)*
`(címke)`	`JMP (vector)`, `JMP (table+2)`	Indirekt ugrás címkén vagy kifejezésen át *(2.1)*
`#<label`	`LDA #<screen_ram`	A címke-cím alsó byte-ja
`#>label`	`LDA #>screen_ram`	A címke-cím felső byte-ja
`*`	`BNE *`	Aktuális program counter (az utasítás saját címe); `*`-gal az elágazások végtelen önhurkot generálnak (offset `$FE`)

Példa — két képernyőlap törlése CONST-tal:

```
; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear
```

A `*` program counter kifejezésekben

A `*` már nem korlátozódik arra, hogy az egész operandus legyen — megjelenhet **bárhol egy operandus-kifejezésen belül**, és annak az utasításnak a címét jelöli, amelyre írták. Fordítási időben oldódik fel az adott utasítás valódi címéhez képest, így nincs szükség címkére a rövid relatív ugrásokhoz vagy a PC-relatív adatolvasásokhoz.

Szintaxis	Példa	Jelentés
`*`	`BNE *`	Ugrás önmagára (végtelen hurok, offset `$FE`)
`*-n` / `*+n`	`BNE *-5`, `BEQ *+4`	Elágazás az aktuális PC-hez képest *n* byte-tal
`JMP *+n`	`JMP *+20`	Az aktuális PC-ből számított abszolút ugrás
`#<*` / `#>*`	`LDA #<*`, `LDA #>*`	Az aktuális PC alsó / felső byte-ja
`#>(*+n)`	`LDA #>(*+63)`	Egy PC-relatív cím alsó / felső byte-ja

PC vs. szorzás. A `*` csak akkor számít program counternek, ha *érték-pozícióban* áll — a kifejezés elején, vagy közvetlenül egy operátor, `(`, `,`, `<`, `>` vagy whitespace után. A szám, `)` vagy azonosító után álló `*` közönséges szorzás, így a `LDA table*2` és a `CONST_A*4` változatlan.

Hol működik. Bármely operandusban, amely már elfogad kifejezést: elágazási célpontok, `JMP` / `JSR`, `LDA` / `STA` /… abszolút és indexelt, immediate alsó/felső-byte operátorok, és a `.assert` kifejezése. A `*` sosem változtatja meg egy utasítás méretét, így minden címzési módban biztonságos.

Lokális (pontos) címkék

Az a címke, amelynek a neve ponttal kezdődik — `.loop`, `.skip`, `.done` — **lokális címke**. A legközelebbi előtte lévő **globális** (nem pontos) címke hatókörébe tartozik, és belsőleg `<globális>.<név>` lesz belőle. Két azonos rövid nevű lokális címke különböző globális címkék alatt **nem** ütközik.

```
DrawSprite:
    LDX #0
.loop:                ; == DrawSprite.loop
    LDA SpriteData,X
    STA $2000,X
    INX
    CPX #63
    BNE .loop         ; a DrawSprite-on belül oldódik fel
    RTS

ClearScreen:
    LDX #0
.loop:                ; == ClearScreen.loop — nincs ütközés
    STA $0400,X
    INX
    BNE .loop
    RTS
```

- Egy hatókörön belül `.név`-ként hivatkozz a lokális címkére.
- Másik hatókörből explicit módon `Globális.név`-ként (pl. `JMP ClearScreen.loop`).
- Az első globális címke előtt írt `.név` sima top-level `.név` marad.

Önmódosító kód operandus-címkék

Tedd egy utasítás operandusa elé a `címke:` prefixet, hogy a címke az **operandus-byte-ra** kerüljön, ne az opcode-ra. Az utasítás abból fordul, ami a kettőspont után áll.

```
setup:
    LDA value:#$00    ; 'value' -> a #$00 operandus-byte címe
    ...
patch:
    LDA #new
    STA value         ; közvetlenül az operandus-byte-ot írja — klasszikus SMC
```

A `value` az `<utasítás címe> + 1`-re mutat (az első operandus-byte), minden címzési módban. Ez leváltja a régebbi `STA instruction+1` / `instruction: LDA #$00` mintát.

---

5. 6502 utasítások

Data Movement

Mnemonik	Leírás	Módok
`LDA`	Akkumulátor betöltése	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`LDX`	X regiszter betöltése	immediate, zeroPage, zeroPageY, absolute, absoluteY

Mnemonik	Leírás	Módok
`LDY`	Y regiszter betöltése	immediate, zeroPage, absolute, absoluteX
`STA`	Akkumulátor tárolása	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`STX`	X regiszter tárolása	zeroPage, zeroPageY, absolute
`STY`	Y regiszter tárolása	zeroPage, absolute

Arithmetic

Mnemonik	Leírás	Megjegyzés
`ADC`	Összeadás carryvel	A legtöbb esetben állítsd be a carryt `SEC`-kel használat előtt
`SBC`	Kivonás carryvel	Állítsd be a carryt `SEC`-kel kivonás előtt
`INC`	Memória növelése	—
`DEC`	Memória csökkentése	—
`CMP`	Összehasonlítás A-val	Flag-eket állít; nem módosítja A-t
`CPX`	Összehasonlítás X-szel	—
`CPY`	Összehasonlítás Y-nal	—

Logic

Mnemonik	Leírás
`AND`	Logikai ÉS az akkumulátorral
`ORA`	Logikai VAGY az akkumulátorral
`EOR`	Kizáró VAGY az akkumulátorral
`BIT`	Bitek tesztelése a memóriában A ellenében (N, V, Z flag-eket állít)

Jumps & Branches

Mnemonik	Leírás
`JMP`	Feltétel nélküli ugrás (abszolút vagy indirekt)
`JSR`	Ugrás szubrutinba (a visszatérési címet a stackre menti)
`RTS`	Visszatérés szubrutinból
`RTI`	Visszatérés megszakításból
`BNE`	Ugrás, ha nem egyenlő (Z=0)
`BEQ`	Ugrás, ha egyenlő (Z=1)
`BCC`	Ugrás, ha carry törölve (C=0)
`BCS`	Ugrás, ha carry beállítva (C=1)
`BMI`	Ugrás, ha negatív (N=1)

Mnemonik	Leírás
`BPL`	Ugrás, ha pozitív (N=0)
`BVC`	Ugrás, ha overflow törölve (V=0)
`BVS`	Ugrás, ha overflow beállítva (V=1)

LBNE / LBEQ / … (Hosszú ugrások)

Nyolc pszeudo-utasítás amelyek feltételes elágazásként viselkednek, de **bármely címet** elérnek, nem csak −128…+127-et. Mindegyik egy invertált elágazásra fordul, amely átugrik egy 3 byte-os `JMP`-t — mindig **5 byte**:

```
LBEQ done      ; erre fordul:   BNE *+3   ($D0 $03)
               ;                JMP done  ($4C lo hi)
```

Hosszú op	Feltétel	Kibocsátott forma
`LBNE`	nem egyenlő (Z=0)	`BEQ *+3 / JMP target`
`LBEQ`	egyenlő (Z=1)	`BNE *+3 / JMP target`
`LBCC`	carry törölve (C=0)	`BCS *+3 / JMP target`
`LBCS`	carry beállítva (C=1)	`BCC *+3 / JMP target`
`LBMI`	negatív (N=1)	`BPL *+3 / JMP target`
`LBPL`	pozitív (N=0)	`BMI *+3 / JMP target`
`LBVC`	overflow törölve (V=0)	`BVS *+3 / JMP target`
`LBVS`	overflow beállítva (V=1)	`BVC *+3 / JMP target`

- Operandus: címke, `*`-kifejezés vagy literál cím — ugyanaz, mint egy normál elágazási célpont.
- Költség: 5 byte és 1 extra ciklus a végrehajtott ágon egy rövid elágazáshoz képest. Nincs automatikus előléptetés rövid elágazásból — te választod az `LBxx`-et explicit módon.
- Amikor egy sima elágazás (`BNE`, `BEQ`, …) tartományon kívülre esik, a fordítói hiba mostantól megnevezi a pontos túllépést, és javasolja a megfelelő `LBxx`-et.

Register Operations

Mnemonik	Leírás
`TAX`	Átvitel A → X
`TAY`	Átvitel A → Y
`TXA`	Átvitel X → A
`TYA`	Átvitel Y → A
`TSX`	Átvitel stack pointer → X
`TXS`	Átvitel X → stack pointer
`INX`	X növelése
`DEX`	X csökkentése

Mnemonik	Leírás
`INY`	Y növelése
`DEY`	Y csökkentése

Shift & Rotate

Mnemonik	Leírás
`ASL`	Aritmetikai balra tolás
`LSR`	Logikai jobbra tolás
`ROL`	Balra forgatás a carryn keresztül
`ROR`	Jobbra forgatás a carryn keresztül

Stack

Mnemonik	Leírás
`PHA`	Akkumulátor push a stackre
`PHP`	Processzor státusz push a stackre
`PLA`	Akkumulátor pull a stackről
`PLP`	Processzor státusz pull a stackről

System / Flags

Mnemonik	Leírás
`CLC`	Carry flag törlése
`CLD`	Decimal mód törlése
`CLI`	Interrupt tiltás törlése
`CLV`	Overflow flag törlése
`SEC`	Carry flag beállítása
`SED`	Decimal mód beállítása
`SEI`	Interrupt tiltás beállítása
`NOP`	Nincs művelet
`BRK`	Kényszerített break / szoftveres megszakítás

Illegal / dokumentálatlan utasítások

Ezek haladó használatra támogatottak. Óvatosan használd — a viselkedés chipenként eltérhet.

`LAX`, `SAX`, `DCP`, `ISC`, `SLO`, `RLA`, `SRE`, `RRA`, `ANC`, `ALR`, `ARR`, `AXS`, valamint az operandust olvasó `NOP` alakok: `NOP #$nn` ($80), `NOP $nn` ($04), `NOP $nn,X` ($14), `NOP $nnnn` ($0C), `NOP $nnnn,X` ($1C)

---

6. Direktívák és makrók — referencia

A direktívák a fordítást vezérlik (címkék, adat, kezdőcím, feltételek, saját makrók); a könyvtári makrók gyakori C64-es feladatokat végeznek el egy sorban — ahelyett, hogy 10–20 utasítást írnál kézzel, egy sort írsz, és az assembler generálja a kódot. Gondolj rájuk beépített szubrutinként.

LABEL

Mint egy **sorszám BASIC-ben** — de szám helyett névvel. Ugrási célpont a `JMP`, `JSR`, `BNE`, stb. számára.

Argumentum	Leírás
Címkenév	Azonosító, amit a `JMP`, `JSR`, `BNE`, stb. használ

Szintaxis:

```
loop:
```

Generált kód:

```
loop:  ; $0820
```

Az aktuális cím megjegyzésként jelenik meg. A címkék **0 byte méretűek**.

COMMENT

Mint a **REM BASIC-ben** — jegyzet magadnak, amit az assembler teljesen figyelmen kívül hagy.

Szintaxis:

```
; Ide jön a megjegyzés szövege
```

Generált kód:

```
; Ide jön a megjegyzés szövege
```

BYTE

Mint a **DATA BASIC-ben** — nyers byte-értékek listáját tárolja inline a programban.

Argumentum	Leírás
Operandus	Vesszővel elválasztott byte-értékek (pl. `$01, $02, $FF` vagy `1, 2, 255`)

Szintaxis:

```
.byte $01, $02, $FF
```

Generált kód:

```
    .byte $01, $02, $FF
```

Lo/hi byte címke-hivatkozások: a BYTE elfogadja a KickAssembler / ca65 stílusú `<label` (alsó byte) és `>label` (felső byte) tokeneket a numerikus értékek mellett. Az assembler fordítási időben feloldja a címke címét, és beszúrja a megfelelő byte-ot. Példa:

```
    .byte <frame_0, >frame_0, <frame_1, >frame_1
```

Ez a `frame_0` címének alsó byte-ját tárolja, majd a felsőt, majd ugyanezt a `frame_1`-re. Hasznos jump table-ök és címlisták építéséhez.

Méret: a lista byte-jainak száma.

WORD

Mint a **DATA BASIC-ben, de 16 bites számokhoz**. Minden érték két byte-ként tárolódik (alsó byte először, majd felső byte — 6502 little-endian sorrend).

Argumentum	Leírás
Operandus	Vesszővel elválasztott 16 bites értékek (pl. `$0400, $C000`)

Szintaxis:

```
.word $0400, $C000
```

Generált kód:

```
    .word $0400, $C000
```

Méret: 2 byte szavanként.

FILL

Mint a `FOR I=1 TO N : POKE addr+I, val : NEXT` — egy memóriablokkot tölt fel ugyanazzal a byte-tal, egyetlen sorban. Kiváló területek törlésére vagy táblák előfeltöltésére.

Argumentum	Leírás
Operandus	`count,value` — pl. `256,0` 256 byte-ot tölt fel nullával

Szintaxis:

```
.fill 256, $00
```

Generált kód:

```
    .fill 256, $00
```

Kifejezés-szintaxis: a `count` és a `value` is elfogad aritmetikai kifejezéseket. Hivatkozhatsz CONST nevekre, használhatsz hex/bináris literálokat, és hívhatsz beépített matematikai függvényeket:

Kifejezés	Jelentés
`TILE_COUNT, $00`	count egy CONST-ból, value hex literál
`40*25, 0`	inline szorzás
`round(sin(PI/4)*255), $80`	trigonometria

Beépített függvények: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, `PI` konstans

Operátorok: `+ - * /` Literálok: `$FF` (hex), `%10110000` (bináris) Alsó/felső byte: `lo(expr)`, `hi(expr)`

Méret: a count érték byte-ban.

ALIGN

Előrébb csúsztatja az aktuális címet a következő tiszta határra nulla kitöltő byte-ok beszúrásával. A C64 megköveteli, hogy a sprite adat 64 byte-os határon kezdődjön — az `ALIGN 64` ezt automatikusan intézi.

Argumentum	Leírás
Határ	Igazítási érték — pl. `64` (sprite határ), `256` (lap), `$2000` (bitmap)

Szintaxis:

```
.align 64
.align $2000
```

Generált kód:

```
    ; ALIGN 64 → $0840 (12 byte kitöltés)
```

Méret: dinamikus — az aktuális program counter pozíciójától függ.

> **Tipp:** használj `ALIGN 64`-et a sprite adat előtt, `ALIGN 256`-ot a lap-igazított táblákhoz.

TEXT

Mint a **PRINT AT** — szöveget ír közvetlenül a C64 képernyőjére adott oszlopban és sorban, a KERNAL használata nélkül. Karakterenként egy LDA/STA párt generál, a `$0400`-nál lévő screen RAM-ot célozva.

Argumentum	Leírás
Text	A megjelenítendő string
X	Oszlop (0–39)
Y	Sor (0–24)
Label (opcionális)	Címkét rendel, amely a kiszámolt képernyőcímre mutat
`lower`	Opcionális jelző — lásd lentebb

Karakterkészlet módok:

A C64-nek két, futásidőben választható karakterkészlete van:

Mód	$D018 bit 1	Nagybetűs bemenet	Kisbetűs bemenet
Nagybetű/grafika (alap)	0	`A` – `Z` → screen code $01–$1A ✓	szintén nagybetűként kezelve
Kisbetű/nagybetű (CHARSET lower után)	1	`A` – `Z` → $01–$1A (nagybetű)	`a` – `z` → $41–$5A (kisbetű) ✓

- **Nagybetűs karakterkészlet (alap):** azt írd, amit nagybetűvel akarsz látni. A `"HELLO"` `HELLO`-ként jelenik meg. A kisbetűs bemenet nagybetűs screen code-okra képződik.
- **Kisbetűs karakterkészlet (`lower`):** pontosan azt a kis/nagybetűzést írd, amit látni akarsz. `"hello"` → kisbetűs megjelenítés, `"HELLO"` → nagybetűs. Futásidejű karakterkészlet-váltást igényel a képernyőre írás előtt (használd a **CHARSET lower** makrót).

Generált kód (nagybetűs mód, `"HELLO"`):

```
    LDA #$08      ; 'H' screen code $08
    STA $0400
    LDA #$05      ; 'E' screen code $05
    STA $0401
    ...
```

Szintaxis:

```
.text 0, 2, "HELLO"           ; nagybetűs karakterkészlet (alap)
.text 0, 2, "hello", lower    ; kisbetűs karakterkészlet
```

A karakterek **screen code-ként** kódolódnak (nem PETSCII). **Méret:** `text.length × 5` byte (LDA + STA karakterenként).

STRING

Mint egy string **POKE-olása** bármely memóriacímre futásidőben. LDA/STA párokat generál, amelyek minden karakter screen code-ját egymást követő címekre másolják.

Argumentum	Leírás
Text	A kiírandó string
Address	Cél memóriacím — `$C000` hex vagy **címkenév**
Label (opcionális)	Címkét rendel, amely a célcímre mutat

Argumentum	Leírás
Shift	Hex érték (00–FF), amely minden screen code byte-hoz hozzáadódik (pl. `$80` = inverz)
`lower`	Opcionális jelző — ugyanaz a szemantika, mint a TEXT-nél (lásd a TEXT szakaszt)

Szintaxis:

```
.string $C000, "HELLO"                  ; nagybetűs karakterkészlet (alap)
.string $C000, "hello", lower           ; kisbetűs karakterkészlet
.string $C000, "HELLO", 80              ; shifttel (inverz)
.string $C000, "hello", 80, lower       ; shift + kisbetű
.string $C000, "HELLO" :my_string       ; macroLabel-lel
```

Generált kód:

```
    LDA #$08      ; 'H' screen code
    STA $C000
    LDA #$05      ; 'E' screen code
    STA $C001
    ...
```

A karakterek **screen code-ként** kódolódnak (nem PETSCII). Az opcionális **Shift** érték minden byte-hoz hozzáadódik, pl. `$80` az inverz videóhoz. **Méret:** `text.length × 5` byte (egy LDA + egy STA karakterenként).

DATA

Mint egy **POKE hurok** — nyers byte-ok listáját írja egy memóriacímre futásidőben, byte-onként egy LDA/STA párral.

Argumentum	Leírás
Bytes	Vesszővel elválasztott byte-értékek
Address	Cél memóriacím — `$C000` hex vagy **címkenév**
Label (opcionális)	Címkét rendel, amely a célcímre mutat

Szintaxis:

```
.data $C000, $01, $02, $03          ; hex cím
.data my_buf, $01, $02, $03         ; címke-cím
.data $C000, $01, $02, $03 :mydata  ; macroLabel-lel
```

Generált kód:

```
    LDA #$01
    STA $C000
    LDA #$02
    STA $C001
    ...
```

Méret: `byte_count × 5` byte (egy LDA + egy STA byte-onként).

> **2.1:** a bájtok címkét is nevezhetnek: a `.data $2010, <target, >target` a `target` címét írja, később definiált címkéét is. A makró `:név` címkéje olyan célcímre is mutathat, amely maga is címke.

RAWBYTES

Mint a **DATA, amely közvetlenül a memóriába töltődik** — egyáltalán nincs futásidejű kód. A byte-ok a PRG betöltésének pillanatától jelen vannak, még mielőtt a kódod elindulna. Ezt használd sprite adathoz, pályatérképekhez, lookup táblákhoz, bármihez, aminek csak egy konkrét címen kell lennie.

Argumentum	Leírás
Bytes	Vesszővel elválasztott byte-értékek
Address	Cél memóriacím — `$C000` hex vagy **címkenév**
Label (opcionális)	Címkét rendel, amely a célcímre mutat

Szintaxis:

```
.rawbytes $C000, $00, $00, $00      ; hex cím
.rawbytes sprite_data, $00, $00     ; címke-cím
.rawbytes $0C50, $00, $00 :nev      ; macroLabel-lel – más kód használhat LDA nev,X-et
```

Méret a kódban: 0 byte. Az adat a megadott címre kerül a kimenetben.

> **DATA vs RAWBYTES:** a DATA LDA/STA kódot generál, amely futásidőben másolja a byte-okat (lassabb, de működik, ha az adatnak dinamikusnak kell lennie). A RAWBYTES egyszerűen elhelyezi a byte-okat — nincs kód, azonnali, nulla költség.

> **2.1:** a bájtok címkét is nevezhetnek (`.rawbytes $2000, <target, >target` — pointertábla); a `:név` címke célcíme lehet címke, később definiált is. Makró puffer-argumentumaként a bájtok saját helyet kapnak a program utáni pufferekben.

RAWTEXT

Mint a RAWBYTES, de szöveghez — a stringet screen code-ként kódolja, és a byte-okat fix címre helyezi **futásidejű kód nélkül**. A szöveg a PRG betöltésének pillanatában készen áll a memóriában.

Argumentum	Leírás
Text	A kódolandó string
Address	Cél memóriacím — `$C000` hex vagy **címkenév**
Label (opcionális)	Címkét rendel, amely a célcímre mutat
Shift	Hex érték (00–FF), amely minden screen code byte-hoz hozzáadódik (pl. `$80` = inverz)
`lower`	Opcionális jelző — ugyanaz a szemantika, mint a TEXT-nél (lásd a TEXT szakaszt)

Szintaxis:

```
.rawtext $C000, "HELLO"                ; nagybetűs karakterkészlet (alap)
.rawtext $C000, "hello", lower         ; kisbetűs karakterkészlet
.rawtext $C000, "HELLO", 80            ; shifttel (inverz)
.rawtext $C000, "hello", 80, lower     ; shift + kisbetű
.rawtext $0400, "HELLO" :my_text       ; macroLabel-lel
```

Generált kód:

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F   ; H E L L O (nagybetűs screen code-ok)

; .rawtext "hello", lower -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; h e l l o (kisbetűs screen code-ok, $41–$5A tartomány)
```

Méret a kódban: 0 byte. Az adat a megadott címre kerül a kimenetben.

> **STRING vs RAWTEXT:** a STRING LDA/STA kódot generál, amely futásidőben másolja a szöveget. A RAWTEXT betöltéskor beépíti a byte-okat a PRG-be — nincs kód, nincs várakozás.

PETSCII

Mint a **RAWBYTES, de KERNAL kimenethez** — a stringet PETSCII byte-ként kódolja (kompatibilis a `$FFD2`-nél lévő CHROUT-tal), és fix címre helyezi futásidejű kód nélkül. Ezt használd, amikor karaktereket akarsz kiírni `JSR $FFD2`-vel egy hurokban; a `.print` ugyanezt a kódolót és `lower` jelzőt használja.

> **PETSCII vs screen code-ok:** a PETSCII és a screen code-ok két különböző kódolás. A screen code `$01` = az A betű; a PETSCII `$41` = az A betű (CHROUT-on át). PETSCII-t csak KERNAL-on keresztüli nyomtatáshoz használj; a screen RAM-ba közvetlen íráshoz használj TEXT/STRING/RAWTEXT-et.

Argumentum	Leírás
Text	PETSCII byte-ként kódolandó string
Address	Cél memóriacím — `$C000` hex vagy **címkenév**
Label (opcionális)	Címkét rendel, amely a célcímre mutat
`lower`	Opcionális jelző — lásd lentebb

Karakterkészlet módok:

Mód	Nagybetűs bemenet (`A – Z`)	Kisbetűs bemenet (`a – z`)
Nagybetű (alap, üres)	`$41` – `$5A` (PETSCII nagybetű CHROUT-on át)	szintén `$41` – `$5A`-ra képezve
Kisbetű (bepipálva)	Az alfabetikus betűk átképződnek, hogy a látható kis/nagybetűzés konzisztens maradjon a kisbetű/nagybetű karakterkészleten	Ugyanaz a szabály

Szintaxis:

```
.petscii $C000, "HELLO"              ; nagybetűs PETSCII (alap)
.petscii $C000, "hello", lower       ; kisbetűs PETSCII ($61–$7A)
.petscii $C000, "HELLO", null        ; null lezáróval
.petscii $C000, "hello", lower, null ; kisbetű + null lezáró
.petscii $C000, "HELLO" :my_msg      ; macroLabel-lel
```

Generált byte-ok (nagybetű, `"HELLO"`):

```
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O (PETSCII $41–$5A tartomány)
```

Méret a kódban: 0 byte. Az adat a célcímre kerül halasztott adat-szegmensként (mint a RAWBYTES-nál).

Null lezáró: a `null` jelzővel egy `$00` byte kerül a szöveg után. Ideális null-lezárt hurkokhoz:

```
    LDX #$00
loop:
    LDA msg,X
    BEQ done        ; a $00 leállítja a hurkot
    JSR $FFD2
    INX
    BNE loop
done:
    RTS
```

Kódolási szabályok:

Bemenet	Nagybetűs mód	Kisbetűs mód
`A` – `Z`	`$41` – `$5A`	`$61` – `$7A`
`a` – `z`	`$41` – `$5A` (kényszerített nagybetű)	`$41` – `$5A`
Szóköz, számjegyek, írásjelek (32–126)	változatlanul	változatlanul
Sortörés	`$0D` (RETURN)	`$0D`
Egyéb	`$20` (szóköz)	`$20`

> **Tipp:** PETSCII-t olyan adathoz használj, amelyet CHROUT-on (`$FFD2`) keresztül fogsz kiírni. A screen RAM-ba közvetlen íráshoz inkább STRING-et vagy RAWTEXT-et.

CHARSET

A VIC-II karakter-ROM-ot váltja nagybetű/grafika mód (C64 alap) és kisbetű/nagybetű mód közt, a `$D018` 1. bitjének futásidejű módosításával.

Argumentum	Leírás
Mód	**Lowercase** — engedélyezi a kisbetű/nagybetű karakterkészletet; **Uppercase** — visszaállítja az alapértelmezett nagybetű/grafika karakterkészletet

Szintaxis:

```
.charset lower    ; váltás kisbetűs karakterkészletre
.charset upper    ; vissza a nagybetű/grafika karakterkészletre
```

Generált kód:

Kisbetűs mód:

```
    LDA $D018
    ORA #$02      ; 1. bit be → kisbetű/nagybetű ROM $1800-nál
    STA $D018
```

Nagybetűs mód:

```
    LDA $D018
    AND #$FD      ; 1. bit törlése → nagybetű/grafika ROM $1000-nál
    STA $D018
```

Méret: 8 byte (LDA abs + ORA/AND imm + STA abs).

Miért ORA/AND közvetlen írás helyett? A `$D018` a screen RAM helyét is vezérli (7–4. bitek). Csak az 1. bit kapcsolása megőrzi a regiszter többi részét.

Tipikus munkafolyamat:

```
.charset lower
.text 0, 0, "hello world", lower
.charset upper
```

> **Megjegyzés:** a CHARSET makró csak a VIC karakter-ROM pointerét változtatja. Nem hívja a `$E544` -et (KERNAL charset init). A legtöbb esetben ez elég; csak akkor hívj előtte `JSR $E544` -et, ha azt akarod, hogy a KERNAL saját print rutinjai is figyeljék a változást.

CHARDEF

Egyetlen 8×8-as egyéni karaktert definiál egy RAM-alapú karakterkészletben. Inline futásidejű kódot bocsát ki, amely 8 byte-ot másol a `base + index * 8` címre végrehajtáskor — nincs szükség előtte címkére vagy `ORG` -ra.

Argumentum	Leírás
Charset base	A RAM karakterkészlet báziscíme (alap `$3800`). Igazítottnak kell lennie, hogy a VIC-II lássa (lásd lentebb).
Character index	Melyik karakter-slotot definiáljuk újra, 0–255. `65` = 'A' az alapértelmezett screen code elrendezésben.
8 byte	Vesszővel elválasztott bitmap sorok, fentről lefelé. Minden byte 7. bitje = a legbaloldalibb pixel.

Szintaxis:

```
.chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00
```

Generált kód (8 × `LDA #b` / `STA target+n` , összesen 40 byte):

```
    LDA #$18
    STA $3A08
    LDA #$3C
    STA $3A09
    LDA #$66
    STA $3A0A
    LDA #$7E
    STA $3A0B
    LDA #$66
    STA $3A0C
    LDA #$66
    STA $3A0D
    LDA #$66
    STA $3A0E
    LDA #$00
    STA $3A0F
```

Célcím: `$3800 + 65 * 8 = $3A08`. Fordítási időben számolódik, és be van égetve az STA operandusokba.

Méret: 40 byte karakterenként (8 × 5).

Tipikus munkafolyamat:

```
    ; VIC-II a $3800-nál lévő RAM charsetre mutasson
    LDA $D018
    AND #$F1
    ORA #$0E       ; 3-1. bitek = %111 → charset a bank+$3800-nál
    STA $D018

    .chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00   ; 'A' újradefiniálása
    .chardef $3800, 66, $7C,$66,$66,$7C,$66,$66,$7C,$00   ; 'B' újradefiniálása
    ; ... további CHARDEF hívások minden egyéni karakterhez
```

Mikor CHARDEF vs alternatívák:

Megközelítés	Használd, ha
CHARDEF	Néhány egyéni karakter kell (mondjuk 1–20). Futásidejű költség 40 byte darabonként.
RAWBYTES @ $3800	Teljes egyéni karakterkészleted van (256 karakter). Összesen 2 KB adat, nincs futásidejű másolás.
INCBIN "charset.bin" @ $3800	Külső karakterkészlet-fájl (a Character Editorral építve). A legtisztább opció.
Charset Canvas + INCBIN	Teljes 256 karakteres bitmap egyetlen 128×128-as képként megfestve.

> **Igazítási emlékeztető:** a VIC-II a charset bázist `$0800` többszörösén várja. Érvényes bankok: `$0000`, `$0800`, `$1000`, … , `$3800` (az aktuális 16 KB-os VIC bankon belül). A RAM charsetek jellemzően a `$2000`, `$2800`, `$3000` vagy `$3800` címen élnek.

BOX_HIT

Tengely-igazított bounding-box (AABB) ütközésteszt két, 4 byte-os zero-page struktúrával leírt téglalap közt. Az eredményt az akkumulátorban adja vissza: **A = 1** átfedésnél, **A = 0** egyébként. Tiszta inline assembly, nincs

szubrutinhívás.

Argumentum	Leírás
Box1 ZP cím	Az első box 4 byte-os struktúrájának zero-page bázisa (alap `$FB`).
Box2 ZP cím	A második box 4 byte-os struktúrájának zero-page bázisa (alap `$F7`).

Struktúra elrendezés (4 byte boxonként, előjel nélküli 8 bites koordináták):

Offset	Mező
`+0`	Left
`+1`	Top
`+2`	Right
`+3`	Bottom

Szintaxis:

```
.box_hit $FB, $F7
```

Generált kód (30 byte, teljesen PC-relatív — nincs szubrutin, nincs abszolút ugrás):

```
    LDA $FD        ; b1 right
    CMP $F7        ; b2 left
    BCC no         ; R1 < L2 → nincs találat
    LDA $F9        ; b2 right
    CMP $FB        ; b1 left
    BCC no
    LDA $FE        ; b1 bottom
    CMP $F8        ; b2 top
    BCC no
    LDA $FA        ; b2 bottom
    CMP $FC        ; b1 top
    BCC no
    LDA #$01       ; találat
    BNE done       ; feltétel nélküli (A ≠ 0)
no: LDA #$00
done:
```

Méret: 30 byte.

Tipikus munkafolyamat:

```
    ; Játékos sprite box a $FB..$FE címen
    LDA sprite_x           ; L
    STA $FB
    LDA sprite_y           ; T
    STA $FC
    CLC
    ADC #23                ; +23 → B (24 pixel magas sprite)
    STA $FE
    LDA $FB
    CLC
    ADC #23                ; R
```

```
    STA $FD

    ; Ellenség box a $F7..$FA címen (hasonlóan feltöltve)
    ; ...

    .box_hit $FB, $F7        ; játékos vs ellenség teszt → A = 0 vagy 1

    CMP #$01
    BNE no_collision
    JSR handle_hit
no_collision:
```

Megkötések:

- Mindkét zero-page címnek `≤ $FC`-nek kell lennie (minden box 4 egymást követő byte-ot igényel: `zp`, `zp+1`, `zp+2`, `zp+3`).
- A koordináták **előjel nélküli 8 biteskénti** kezelődnek (0–255). Ezen tartományon kívüli előjeles sprite-koordinátáknál tárolás előtt normalizálj.
- A két box átfedhet a ZP térben, ha akarod, de általában 8 különálló byte-ot szeretnél.

Miért nem szubrutin? Az inline generálás elkerüli a JSR/RTS overhead-et (14+ ciklus), és a tesztet a cache-ben tartja a szoros game hurkokhoz. Ha sok párt kell tesztelned, tegyél kézzel egy saját `JSR box_hit_sub`-ot egyetlen BOX_HIT blokk köré.

Összehasonlítás a UB `box_hit()` függvényével: az Ultimate Basic ugyanezt a 6502 logikát csomagolja futásidejű függvényként, amely egy változóba tér vissza. VA-ban a BOX_HIT-et inline helyezed oda, ahol a tesztre szükség van; az eredmény az `A`-ban van.

INCBIN

Mint a **BLOAD BASIC-ben** — beemel egy külső binárisfájlt (`.bin`, `.prg`, `.sid`, `.raw`), és közvetlenül beágyazza a lefordított PRG-be a megadott címre.

Argumentum	Leírás
File	Tallózz egy `.bin`, `.prg`, `.sid` vagy `.raw` fájl kiválasztásához
Address	Cél betöltési cím (pl. `$C000`)

Szintaxis:

```
.incbin "music.bin", $C000
```

Generált kód megjegyzés:

```
    ; INCBIN "music.bin" @ $C000 (2048 byte)
    .byte $01, $02, ...
```

Méret a kódban: 0 byte (halasztott adat-szegmens). A bináris a megadott címre ágyazódik.

KOALA

Egy **Koala Painter** képet (`.koa` / `.kla` , 160×200 multicolor) tölt be és jelenít meg. A képadat az `INCBIN` -hez hasonlóan fix címre ágyazódik (a fájl 2 bájtos `$6000` betöltési címét levágja), maga a direktíva pedig egy 86 bájtos megjelenítő rutinra fordul, ami a blokk helyén fut.

Argumentum	Leírás
File	Tallózz egy `.koa` / `.kla` fájlt (10003 bájt, vagy 10001 betöltési cím nélkül)
Address	A bitmap címe: `$6000` (alapértelmezett, Koala natív, VIC bank 1, screen RAM `$4400`) vagy `$2000` (VIC bank 0, screen RAM `$0400`)

A megjelenítő rutin a screen RAM részt a képernyőre, a color RAM részt a `$D800` -ra másolja, a háttér bájtot a `$D021` -be írja, a `$DD00` -ban kiválasztja a VIC bankot (a port többi bitjét megtartva), és bekapcsolja a multicolor bitmap módot (`$D011 = $3B` , `$D016 = $18` , `$D018 = $18`). Más címet elutasít: a többi 8K-s terület a BASIC/KERNAL ROM alatt van (a másoló ciklus a ROM-ot olvasná), vagy a karakter-ROM árnyékba esik.

Szintaxis:

```
.koala "picture.koa"
.koala "picture.koa", $2000
```

Példa — kép megjelenítése és várakozás:

```
* = $0801
.koala "picture.koa"
wait:
    JMP wait
```

Méret a kódban: 86 byte (megjelenítő rutin); a 10001 bájt képadat a megadott címre kerül. A C64 Visual Assembler Hires szerkesztője kompatibilis fájlt exportál.

LEVEL / LEVEL_ROW

Egy **Pálya szerkesztő** projektet tesz a programba, és pályasorokat másol egy képernyő-pufferbe — ezek egy görgetős játék építőkövei. A C64 Visual Assembler `level-demo.asm` mintája egy teljes, joystickkal görgetett pálya.

LEVEL

INCBIN-szerű direktíva, ami **0 byte kódot** generál. A Pálya szerkesztő projektből (a C64 Visual Assembler Pálya szerkesztője által mentett `.bin`) csak az **elhelyezett** képernyők kerülnek a memóriába a megadott címtől (alapértelmezés `$2000`), hely-sorrendben, képernyőnként 2000 bájt (1000 screen code + 1000 szín). Az azonos képernyők egyszer tárolódnak, az üres helyek egy közös üres képernyőn osztoznak. A képernyők után jönnek a pointer-táblák. Egy pálya legfeljebb 7 × 10 képernyős lehet.

Szintaxis:

```
.level "level.bin"
.level "level.bin", $4000
```

Definiált név	Jelentés
`LEVEL_SLOT_LO` / `LEVEL_SLOT_HI`	Az egyes helyek képernyőjének címe (soronként, soronként `LEVEL_W` hely)
`LEVEL_ROW_SLOT`	Pálya karaktersor → a képernyősorának első helye
`LEVEL_ROW_LO/HI`, `LEVEL_CROW_LO/HI`	Pálya karaktersor → a screen code / szín sorának eltolása a képernyőn belül
`LEVEL_COL_SLOT`, `LEVEL_COL_OFS`	Nézet kezdőoszlop → képernyőoszlop és oszlop a képernyőn belül
`LEVEL_W`, `LEVEL_H`	Pályaméret képernyőkben
`LEVEL_COLS`, `LEVEL_ROWS`	Pályaméret karakterekben
`LEVEL_MAX_X`, `LEVEL_MAX_Y`	Egy 40×25-ös nézet legnagyobb pixeles görgetési pozíciója
`LEVEL_BG`, `LEVEL_MC1`, `LEVEL_MC2`, `LEVEL_MULTICOLOR`	A pálya háttér- és multicolor színei, módja
`LEVEL_DATA`, `LEVEL_SIZE`, `LEVEL_END`	A pályaadat kezdete, mérete és az utána következő első szabad cím
`LEVEL_OBJ_X_LO/HI`, `LEVEL_OBJ_Y_LO/HI`, `LEVEL_OBJ_TYPE`, `LEVEL_OBJ_PARAM`	Objektumtábla (a Pálya szerkesztő objektumrétege): pozíció pálya-pixelben, típus és paraméterbájt, görgetési sorrendben — `LEVEL_OBJ_COUNT` elem (max. 255)
`LEVEL_OBJ_T_START` … `LEVEL_OBJ_T_TRIGGER`	Objektumtípus-számok: START 0, ENEMY 1, ITEM 2, DOOR 3, EXIT 4, TRIGGER 5
`LEVEL_START_X`, `LEVEL_START_Y`	Az első Kezdőpont objektum pozíciója (csak ha van ilyen a pályán)
`LEVEL_ATTR`, `LEVEL_ATTR_SOLID` … `LEVEL_ATTR_COLLECT`	Karakter-attribútum tábla (screen code-onként egy jelzőbájt; csak ha a pályán van attribútum) és a bitek: 1, 2, 4, 8
`LEVEL_BORDER`, `LEVEL_SCR_COLORS`, `LEVEL_SCR_BG/BORDER/MC1/MC2`	A pálya kerete; helyenkénti színtáblák (soronként, mint a `LEVEL_SLOT_*`), ha egy képernyőnek saját színei vannak (`LEVEL_SCR_COLORS` = 1)

Tipikus védelem: `.assert LEVEL_END <= $9200, "level data runs into the code"`.

LEVEL_ROW

A pálya egy 40 karakteres sorát másolja egy pufferbe: az **X**-ben megadott pályasort, a `col` változóban tárolt pályaoszloptól kezdve, a `dst` pointerben (`dst` / `dst+1`, lo/hi) tárolt címre. A harmadik, `color` argumentummal a screen code-ok helyett a színeket másolja (color RAM-hoz). A két képernyőre átnyúló 40 karakteres ablakot önmódosító másolóciklus kezeli (~14 ciklus/bájt). Az X megmarad, az A és az Y elvész. Kell hozzá egy `.level` blokk a programban.

Szintaxis:

```
.level_row dst_lo, view_x
.level_row dst_lo, view_x, color
```

Méret a kódban: 132 byte — ha egy elágazásnak át kell ugrania, használj `LBEQ` / `LBNE`-t.

SID

Mint a **BLOAD zenéhez** — betölt egy `.sid` fájlt a PRG-be, és automatikusan kiolvassa az Init és Play címeket a fejlécből. Hívd az Init-et egyszer induláskor, majd hívd a Play-t az IRQ kezelődből minden frame-ben.

Argumentum	Leírás
File	Tallózz egy `.sid` fájl kiválasztásához
Custom address (opcionális)	Felülírja a SID natív betöltési címét (pl. `$1000`). Hagyd üresen a SID fejlécből származó cím használatához.

A SID fejlécéből:

- **Title / Author** a SID fejlécből
- **Load address** — hova kerül az adat a memóriában (a felülírás utáni tényleges cím)
- **Init address** — ezt hívd JSR-rel a zene inicializálásához (relokáció esetén korrigálva, ha egyéni címet használsz)
- **Play address** — ezt hívd JSR-rel minden frame-ben egy IRQ kezelőben (relokációhoz korrigálva)
- Egy **(relocated)** jelvény jelenik meg, amikor egy egyéni cím elmozdítja az adatot az eredeti pozíciójából

Szintaxis:

```
.sid "Ikari_Warriors.sid"
.sid "Ikari_Warriors.sid", $1000
```

Generált kód megjegyzés:

```
    ; SID "Ikari_Warriors.sid" @ $1000  Init:$1000  Play:$1006  (4096 byte)
```

Méret a kódban: 0 byte inline. A SID bináris a megadott címre kerül halasztott chunkként a PRG-ben.

> **Fontos:** a legtöbb SID fájl beégetett belső abszolút címeket tartalmaz. Csak akkor relokálhatók, ha az egész bináris ugyanazzal az offszettel tolódik el. Ha egy SID belső ugrásokat tartalmaz `$10xx`-re, akkor `$1000`-en kell maradnia — más címre mozgatva ezek a belső hivatkozások elromlanak.

> **Tipikus használat:** adj a SID-nek saját címet (`.sid "tune.sid", $1000`), ha az eredeti ütközik a kódoddal. Hívd az Init-et egyszer induláskor, majd hívd a Play-t minden frame-ben egy raszter IRQ kezelőből — vagy bízd mindkettőt a SID_PLAYER makróra.

Címkék: a SID direktíva megadja a `SID_LOAD` (ahova az adat kerül), a `SID_INIT` és a `SID_PLAY` címkét (az init és a play címe, egyedi címnél azzal együtt eltolva). A `SID_PLAY` hiányzik, ha a zenének nincs play címe (saját IRQ-t telepítő RSID). A saját kódodban is használhatod őket, pl. `JSR SID_PLAY` egy IRQ kezelőben.

SID_PLAYER

Egyetlen sorral lejátssza a **SID** direktíva zenéjét: inicializálja, majd egy raszter interruptból folyamatosan hívja a play rutinját, így a programod közben a saját dolgát végezheti.

Argumentum	Leírás
Song	Az init rutinnak A-ban átadott subtune száma (alapból `0`)
Raster	Az interrupt rasztersora, `$00` – `$FF` (alapból `$FA`)

Mit csinál: `SEI`, `LDA #song` / `JSR SID_INIT`, kikapcsolja a CIA1 időzítő IRQ-t (`$DC0D`), az IRQ vektort (`$0314/$0315`) a beépített handlerére állítja, beállítja a rasztersort (`$D012`, a `$D011` 8. bitje törölve) és a raszter IRQ-t (`$D01A`), `CLI`, majd átugorja a handlert. A handler nyugtázza az interruptot (`ASL $D019`), meghívja a `JSR SID_PLAY`-t, és a KERNAL-ban folytatja (`JMP $EA31`, ez a billentyűzetet is olvassa).

Szintaxis:

```
.sid "Ikari_Warriors.sid"
.sid_player
.sid_player 1, $80
```

Méret a kódban: 58 byte.

> **Megjegyzés:** kell hozzá egy `.sid` sor a programban (`SID_INIT` és `SID_PLAY`), különben a fordítás hibával megáll. A CIA1 időzítő IRQ helyett a KERNAL képkockánként egyszer, a raszter IRQ-ból fut — a billentyűzet és a `GETIN` továbbra is működik.

INCLUDE

Mint a **MERGE BASIC-ben** — beemel egy másik fájlt, és inline kifejti ezen a pozíción. Kiváló újrahasználható szubrutin-könyvtárakhoz.

Kétféle fájl támogatott:

- **Sima assembly forrás** (`.inc`, `.asm`, `.s`) — VMA forrásként olvasódik. Minden fordításkor újra beolvasódik, így bármilyen szerkesztővel szerkesztheted.
- **Visual Assembler projekt** (`.json`) — a C64 Visual Assembler blokkprogramjának blokkjai.

Argumentum	Leírás
File	Egy `.inc` / `.asm` / `.s` forrás vagy egy `.json` Visual Assembler projekt
Load address (opcionális)	Ha beállított (hex, pl. `C000`), a beemelt kód arra a címre kerül — egy `ORG` kerül elé, a beemelt fájl saját ORG-jai pedig nem számítanak. Nélküle a beemelt fájl saját ORG-jai döntik el az elhelyezést.

Szintaxis:

```
.include "library.json"
.include "macros.inc", $1500
.include "sprites.asm"
```

- Add meg a **fájlkiterjesztést** — egy csupasz név, mint a `.include "macros"`, `.include "macros.json"`-ként kezelődik.
- Útvonal-feloldás: először a forrás mellett, majd minden keresési mappában (a CLI `--base-dir` / `--search` kapcsolói); egy más gépen írt útvonal a puszta fájlnévre esik vissza. A nem található include nem hiba — a fordítás nélküle megy tovább.

Generált kód (nincs cím-felülírás):

```
    ; .include "library.json"
   ... (a beemelt kód következik)
```

Generált kód (`C000` betöltési címmel):

```
    ; .include "library.json" @ $C000
   *=$C000
   ... (a beemelt kód következik)
```

> **Tipp:** használd az INCLUDE-ot újrahasználható szubrutin-könyvtárak építésére, amelyeket megoszthatsz projektek közt. Az `.inc` / `.asm` / `.s` fájlok akkor a legjobbak, ha sima szövegszerkesztőben akarod szerkeszteni a könyvtárat, vagy más 6502 assemblerekkel megosztani; a `.json` akkor, ha a könyvtár Visual Assembler blokkprogram. Állíts be betöltési címet, ha a könyvtárnak nincs saját ORG-ja, vagy ha felül akarod írni az alapértelmezett elhelyezését.

> **2.1:** a nem található include hiba, és az is, ha egy fájl önmagát emeli be (közvetlenül vagy mélyebben). A beemelt fájl saját `.include`, `.incbin` és `.sid` sorai is betöltődnek a saját mappájához képest, és a makróhívásai is kibomlanak — legfeljebb 16 szint mélyen.

TABLE

Mint a **DIM egy konkrét címen** — nevet ad egy lookup táblának, és beállítja, hol él a memóriában. Tegyél utána `.byte`, `.word` vagy `.fill` sorokat a tábla tartalmának definiálásához.

Argumentum	Leírás
Name	Címke-azonosító a táblához (pl. `color_table`)
Address	Fix cím, ahol a tábla kezdődik (pl. `$C000`)

Szintaxis:

```
.table color_table $C000
.table lookup              ; az aktuális címen
```

Generált kód:

```
color_table:
```

Címmel a program counter oda ugrik; cím nélkül a tábla az aktuális címen kezdődik. Tegyél `.byte` / `.word` / `.fill` sorokat utána a tartalom feltöltéséhez.

Méret: 0 byte.

ORG

Beállítja, hova kerül a program (vagy egy szakasza) a memóriában — mintha egy kezdőcímet választanál, mielőtt gépi kódot gépelnél. Minden programnak legalább egy ORG kell. A szabványos C64 BASIC-ből betölthető kezdet a `$0801`.

Argumentum	Leírás
Address	Az új origin cím (pl. `0801` HEX-ben, vagy `2049` DEC-ben)

Argumentum	Leírás
HEX / DEC	A cím-bemenet váltása hexadecimális és decimális megjelenítés közt

Szintaxis:

```
* = $C000
```

Generált kód:

```
* = $C000
```

Méret: 0 byte. Az ORG maga nem generál gépi kódot.

Minden ORG új szakaszt kezd. Az utána következő sorok arról a címről kezdve assembleálódnak. A PRG minden szakaszt egyetlen fájlban tartalmaz — a szakaszok közti rések nullákkal töltődnek fel.

Példa — kód a `$0801`-en, adattábla a `$C000`-en:

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

> **Tipp:** ORG nélkül a program a `$0801`-en, a BASIC elején kezdődik. C64 BASIC-ből betölthető programnál a tipikus kezdőcím a `$0801` (2049 decimális). Ha a **BASIC SYS stub** engedélyezett, az assembler egy rövid BASIC sort tesz a `$0801`-re, és a kódod a `$080D`-n kezdődik.

LOOP / NEXT

Mint a `FOR X=N TO 1 STEP -1 : ... : NEXT X` BASIC-ben — N-től 1-ig számol lefelé az X vagy Y regiszterrel. Tedd az utasításaidat a LOOP és a NEXT közé, és automatikusan a helyes számú alkalommal hurokban fut.

LOOP

Argumentum	Leírás
Register	`X` vagy `Y` — a számláló regiszter
Count	Ciklus-iterációk száma (hex vagy decimális, pl. `0A` = 10)
Label	Automatikusan generált loop címke (pl. `loop0`)

Szintaxis:

```
.loop X, 10, loop0
```

Generált kód:

```
    LDX #$0A
loop0:
```

Méret: 2 byte (LD_ opcode + immediate operandus).

NEXT

Argumentum	Leírás
Register	Automatikusan a LOOP regiszteréhez illesztve
Label	Automatikusan a LOOP címkéjéhez kötve

Szintaxis:

```
.next loop0
```

Generált kód:

```
    DEX
    BNE loop0
```

Méret: 3 byte (DEX + BNE + branch offset).

Példa — 10 képernyőcella törlése:

```
    LDX #$0A
loop0:
    LDA #$20        ; szóköz karakter
    STA $0400,X
    DEX
    BNE loop0
```

FOR / ENDF

Mint a `FOR X=0 TO N-1 : ... : NEXT X` BASIC-ben — 0-tól számol *felfelé*. Ideális, amikor előre haladó indexre van szükséged, pl. egy string vagy tömb végigjárásához.

FOR

Argumentum	Leírás
Register	`X` vagy `Y` — a számláló regiszter
Count	Ciklus-limit (hex vagy decimális, pl. `$12` = 18). Az X/Y 0-tól limit-1-ig fut.
Label	Automatikusan generált loop címke (pl. `for0`)

Szintaxis:

```
.for X, $12, for0
```

Generált kód:

```
    LDX #$00
for0:
```

Méret: 2 byte (LD_ opcode + `#$00`).

ENDF

Argumentum	Leírás
Register	Automatikusan a FOR regiszteréhez illesztve
Label	Automatikusan a FOR címkéjéhez kötve
Count	Automatikusan a párosított FOR-ból másolva

Szintaxis:

```
.endf for0
```

Generált kód:

```
    INX
    CPX #$12
    BNE for0
```

Méret: 5 byte (IN_ + CP_ #imm + BNE offset).

Példa — null-lezárt string kiírása:

```
    LDX #$00
for0:
    LDA msg,X       ; msg = PETSCII string fix címen
    BEQ done        ; null lezáró → kilépés
    JSR $FFD2       ; CHROUT
    INX
    CPX #$12        ; max 18 karakter
    BNE for0
done:
    RTS
```

> **LOOP vs FOR:** a LOOP lefelé számol (N→1) — jó késleltetésekhez, kitöltésekhez, pixel hurkokhoz. A FOR felfelé számol (0→N) — jó string/tömb eléréshez. Mindkettő használhat X-et vagy Y-t.

PUSH / PULL

Mint **változók mentése egy GOSUB előtt és visszaállítása utána** — de a 6502 hardveres stacket használja. Ha egy szubrutin használja A-t, X-et vagy Y-t, vedd körül PUSH-sal és PULL-lal, hogy a hívó kód regiszterei megmaradjanak.

PUSH

Egy vagy több regisztert push-ol a stackre. A sorrend mindig A → X → Y (legbelső először).

Argumentum	Leírás
Registers	Bármely kombináció: `A`, `X`, `Y`, `AX`, `AY`, `XY`, `AXY`

Szintaxis:

```
.push AXY
```

Generált kód (példa: `AX`):

```
    PHA
    TXA
    PHA
```

Méret: 1 byte A-hoz (`PHA`), 2 byte X-hez vagy Y-hoz (transfer + push).

PULL

Visszaállítja a regisztereket a stackről **fordított sorrendben** (Y → X → A).

Argumentum	Leírás
Registers	Ugyanaz, mint a PUSH — meg kell egyeznie a megfelelő PUSH-sal

Szintaxis:

```
.pull AXY
```

Generált kód (példa: `AX`):

```
    PLA
    TAX
    PLA
```

> **Szabály:** a PUSH és PULL mindig **azonos regiszterkészletet** kell használjon. `PUSH AX` → `PULL AX` (belsőleg fordítva állít vissza: előbb X, majd A).

END / RTS alias

Mint az **RTS barátságosabb makró-névvel** — a `.end` egyetlen `RTS` byte-ot bocsát ki, rövid szubrutin-lezáróként.

Szintaxis:

```
.end
```

Generált kód:

```
    RTS
```

Méret: 1 byte.

Ezt akkor használd, ha egy szubrutin-vége jelölőt szeretnél, ami kicsit inkább makrónak, mint nyers utasításnak olvasható.

MACRO / ENDM / INVOKE

Mint **egy nevesített GOSUB paraméterekkel** — definiálj egyszer egy újrahasználható kódrészletet (MACRO…ENDM), majd hívd bárhol INVOKE-kal. Adj át minden alkalommal más argumentumértékeket ahelyett, hogy kódot másolgatnál.

MACRO (definíció eleje)

Argumentum	Leírás
Name	A makró azonosítója (pl. `setColor`)
Params	Opcionális, vesszővel elválasztott paraméternevek (pl. `color` vagy `color, count`)

Jelöli a makró-definíció elejét. A MACRO és ENDM közti sorok a makró teste. A paraméteres makró **nem generál kódot** ott, ahol a definíció van; a paraméter nélküli makró ott szubrutinként is lefordul (`JSR név` hívja). A törzsben a paraméter `{paramName}` alakban vagy puszta névvel írható; a hívás behelyettesíti az argumentumot — az utasítások operandusában és a könyvtári makrók argumentumaiban is (`.set_border c`, `.sprite_pos n, x, y` …). A törzs címkéi, LOOP-címkéi és futásidejű `.if` / `.while` címkéi hívásonként egyediek. Makrótörzsben makróhívás nincs kibontva (hiba).

Generált kód:

```
; .MACRO setColor (color)
    ... (törzs)
; .ENDM
```

Szintaxis:

```
.macro setColor color
    LDA {color}
    STA $D020
.endm
```

ENDM (definíció vége)

Lezárja az aktuális makró-definíciót. Nincs argumentuma.

INVOKE

Meghív egy definiált makrót ezen a pozíción, és behelyettesíti a megadott argumentumértékeket a test `{paramName}` helyőrzőibe.

Argumentum	Leírás
Macro name	A programban vagy egy beemelt fájlban definiált makró
Arguments	Vesszővel elválasztott argumentumértékek, amelyek illeszkednek a makró paraméterlistájához (pl. `#$07`)

Generált kód:

```
; .invoke setColor(#$07)
    LDA #$07
    STA $D020
```

Szintaxis:

```
; egyetlen argumentum:
.invoke setColor(#$07)

; több argumentum:
.invoke drawPixel($10, $20)

; nincs argumentum:
.invoke clearScreen

; szöveg / string argumentumok (idézőjelben):
.invoke printText("Hello, World!")

; .call alias (a .invoke szinonimája):
.call setColor(#$07)
```

A makró teste inline kifejtődik, a `{paramName}` a tényleges argumentumokra cserélve. A szóközzel elválasztott forma (`.invoke setColor #$07`) is elfogadott.

Argumentum-típusok:

- **Numerikus**: `#$07`, `$10`, `255` — hex vagy decimális értékek
- **Szöveges stringek**: `"Hello, World!"` — idézőjeles stringek; az idézőjeleken belüli vesszők a szöveg részének számítanak, nem argumentum-elválasztónak
- **Vegyes**: `#$07, "hello", $20` — bármely kombináció

> **Tipp:** definiáld a makrókat a programod tetején (vagy alján), majd INVOKE-old őket, ahol szükséges. A makrók többször is meghívhatók különböző argumentumokkal.

> **2.1:** egy makró másik makrót is hívhat (a `both(a, b)` a `setc(a)` -t); a belső hívás a külső hívás argumentumait kapja, saját egyedi lokális címkékkel. Az önmagát hívó makró 16 szint után hibával áll meg, nem akad el.

REGION / ENDREGION

Csak csoportosítás a szerkesztőknek és a listáknak — **nulla byte**, nulla hatás a lefordított kódra.

Argumentum	Leírás
Region name	Szabad szöveges címke a szakaszhoz (pl. `init`, `game_loop`, `sprite_setup`)

Szintaxis:

```
.region init
    ; kód...
.endregion
```

Generált kód:

```
; region init
    SEI
    LDA #$00
    STA $D020
; endregion init
```

Méret: 0 byte a REGION és az ENDREGION esetén is.

> **Megjegyzés:** a régiók **egymásba ágyazhatók**. Minden ENDREGION a legközelebbi nyitott REGION-t zárja. Nincs hatás a lefordított kimenetre.

DEFINE / IF / ELSE / ENDIF

Mint **egy kapcsoló, amit az assembler olvas** — a `DEFINE DEBUG` bekapcsol egy szimbólumot, majd minden `IF DEBUG` sorai lefordulnak, és az `ELSE` ága kimarad. Vedd ki a DEFINE-t, és az IF rész eltűnik a kimenetből. Release buildekhez nem kell kódot törölni.

DEFINE

Argumentum	Leírás
Symbol	Egy vagy több, vesszővel elválasztott aktiválandó azonosító (pl. `DEBUG` vagy `DEBUG, PAL`)

Szintaxis:

```
.define DEBUG, PAL
```

Generált kód:

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

Egy `DEFINE` egyszerre több szimbólumot is aktiválhat (vesszővel elválasztva). Tedd a DEFINE-okat a programod tetejére.

IF

Argumentum	Leírás
Condition	Tesztelendő azonosító (egy `DEFINE` szimbólumhoz kell illeszkednie ahhoz, hogy aktív legyen)

Szintaxis:

```
.if DEBUG
```

Generált kód:

```
; .IF DEBUG
```

Az `IF` és `ENDIF` (vagy `ELSE`) közti sorok a feltételtől függően fordulnak le vagy maradnak ki. A puszta név igaz, ha `.define`-olt vagy nem nulla `.const`, hamis, ha egyik sem; minden más feltétel konstans-kifejezés kell legyen — ami fordításkor nem értékelhető ki, hiba. A kimaradt sorok `; [IF skipped] …` megjegyzésként jelennek meg, és **nulla byte-ot** generálnak.

ELSE

Nincs argumentuma. Jelöli az alternatív ágat — akkor assembleálódik, amikor az `IF` feltétel *nem* aktív.

Szintaxis:

```
.else
```

Generált kód:

```
; .ELSE
```

ENDIF

Nincs argumentuma. Lezárja a feltételes részt.

Szintaxis:

```
.endif
```

Generált kód:

```
; .ENDIF
```

Méret: 0 byte mind a négy direktívánál. Csak a *közöttük* lévő tartalom számít.

Példa — debug keret-villantás, a release build kihagyja:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; piros keret
    STA $D020
; .ELSE
```

```
    LDA #$00        ; fekete (release)
    STA $D020
; .ENDIF
```

Példa — több szimbólum egy DEFINE-ban:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL időzítési konstans
; .ELSE
    LDA #$xx        ; NTSC időzítési konstans
; .ENDIF
```

Az egymásba ágyazott `IF`-ek támogatottak. Ha egy külső rész kimarad, a belsők is kimaradnak.

> **Megjegyzés:** ez fordítási idejű feltételkezelés. Futásidejű compare/branch cukorért lásd a **Futásidejű IF / ELSE / ENDIF**-et lentebb.

.ASSERT

Egy **fordítási idejű józansági ellenőrzés**. A `.assert` egy kifejezést értékel ki assembleálás közben; ha hamis (`0`), a build egy beszédes hibával leáll, amely tartalmazza a tényleges értéket. Ha igaz (nem nulla), semmit nem bocsát ki.

Argumentum	Leírás
Expression	Bármely assembler-kifejezés: címkék, `CONST`-ok, `*` (program counter), aritmetika és összehasonlítások (`<` , `<=` , `>` , `>=` , `==` , `!=`)
Message	Opcionális szöveg, amely a hibaüzenethez fűződik

Szintaxis:

```
.assert spriteData < $C000
.assert * < $A000
.assert end - start <= 256, "a sprite tábla túllépett egy lapot"
```

Viselkedés:

- **Méret:** 0 byte.
- Egy hamis assertion megszakítja az assembleálást: `` `.assert end - start <= 256` hamis (érték: 0). a sprite tábla túllépett egy lapot ``
- Az a kifejezés is hibára fut, amelyet nem lehet kiértékelni (nem definiált címke, stb.), *„nem értékelhető ki fordítási időben*" üzenettel.
- Az összehasonlítások `1` / `0`-t adnak; tedd a `.assert`-et bárhova a program folyamában — azon a címen ellenőrződik, ahol áll, így a `.assert * < $D000` az aktuális kimeneti pozíciót teszteli.

Példa — sprite-adat védése lap-átlépés ellen:

```
* = $3000
sprite_data:
    .byte $00, $00, $00        ; … 63 byte …
sprite_data_end:
```

```
    .assert sprite_data_end - sprite_data == 63, "a sprite pontosan 63 byte kell legyen"
    .assert (sprite_data % 64) == 0, "a sprite 64 byte-ra igazított kell legyen"
```

CONST

Mint **egy nevesített változó, amely sosem változik** — `SCREEN = $0400`. Használd a nevet ahelyett, hogy mindenhol nyers címeket gépelnél, így a kód olvashatóbb és később könnyebben módosítható.

Argumentum	Leírás
Name	A konstans azonosítója (pl. `SCREEN`)
Value	Numerikus érték a választott számrendszerben (pl. `0400` HEX-ben = `$0400` cím), vagy egy PC-relatív kifejezés (lásd lentebb)
Format	HEX vagy DEC — vezérli, hogyan adod meg és jelenik meg az érték

Szintaxis:

```
.const SCREEN = $0400
.const FRAMES_1S = 60
```

Generált kód:

```
; .CONST SCREEN = $0400
```

PC-relatív kifejezések (`*+N` / `*-N`):

Az érték elfogad `*+N`-t vagy `*-N`-t is, ahol a `*` magának a `.const` sornak a címe. Ezzel egy nevesített aliast hozhatsz létre egy közeli utasításon belüli byte-hoz — a klasszikus önmódosító kód minta:

```
.const op = *+1      ; op → a következő LDA immediate operandusának címe
LDA #$00             ; a $00-t futásidőben módosítjuk
...
STA op               ; felülírja a #$00 byte-ot → az LDA legközelebb az új értéket olvassa
```

A CONST 0 byte-ot bocsát ki; a címke fordítási időben `aktuális_cím + 1`-re oldódik.

Aritmetikai kifejezések:

Az érték elfogad általános aritmetikát, beleértve a korábban definiált CONST nevekre való hivatkozásokat, hex/bináris literálokat és beépített matematikai függvényeket:

```
.const SCREEN      = $0400
.const SCREEN_END  = SCREEN + 40*25   ; 1000 byte-tal később
.const COLOR_RAM   = $D800
.const MID_X       = 160
.const SIN_TABLE   = round(sin(PI/8) * 127)   ; előre kiszámolt szinuszérték
```

Beépített függvények: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, `PI` konstans

Operátorok: `+ - * /` Literálok: `$FF` (hex), `%10110000` (bináris) Alsó/felső byte: `lo(expr)`, `hi(expr)`

Méret: 0 byte.

VAR

Mint a **CONST, de automatikusan foglalva** — a `VAR` zero-page tárhelyet foglal egy címkéhez anélkül, hogy be kellene gépelned a címet. Használd számlálókhoz, pointerekhez és rövid életű, ZP-be való állapothoz.

Argumentum	Leírás
Name	Változónév / címke
Size (opcionális)	Foglalandó byte-ok száma. Hagyd el egy byte-hoz.

Szintaxis:

```
.var counter
.var timer, 2
.var lives
```

Gyakorlati példa:

```
.region Vars
.var counter
.var timer, 2
.endregion

LDA #$00
STA counter
```

Generált kód:

```
; .var counter
```

Méret: alapból 1 byte, vagy `N` byte, ha megadod a méretet.

Az allokátor egy konfigurálható zero-page kurzort jár be (`$02` -tól `$FE` -ig), és a következő szabad slotot rendeli hozzá. Ha a kért terület átfed egy már használt címkével, a fordító figyelmeztetést ad.

Futásidejű IF / ELSE / ENDIF

Mint **egy valódi elágazás-sablon** — ez a változat futásidőben működik, nem fordítási időben. Összehasonlítja `A` , `X` vagy `Y` értékét egy immediate értékkel, és a helyes `CMP` / `CPX` / `CPY` + elágazás szekvenciát bocsátja ki neked.

Argumentum	Leírás
Register	`A` , `X` vagy `Y`
Operator	`==` , `!=` , `<` , `<=` , `>` , `>=`
Value	Immediate érték HEX vagy DEC formátumban

Szintaxis:

```
.if A == #$10
    LDA #$07
.else
    LDA #$0F
.endif
```

Méret: a választott elágazásoktól és az összehasonlítási formától függ.

Az összehasonlítás alapból előjel nélküli. A `<=` és `>` esetén a makró a kiválasztott regiszterhez a legrövidebb ekvivalens elágazás-láncra fejt ki.

WHILE / ENDW

Mint **egy futásidejű hurok teszttel a tetején** — a test addig fut, amíg a feltétel igaz marad.

Argumentum	Leírás
Register	`A`, `X` vagy `Y`
Operator	`==`, `!=`, `<`, `<=`, `>`, `>=`
Value	Immediate érték HEX vagy DEC formátumban

Szintaxis:

```
.while A != #$00
    JSR getchar
.endw
```

Méret: a hurok testétől és az összehasonlítási formától függ.

Használd a `WHILE`-t, ha a hurok az első iteráció befejezése előtt is véget érhet. Ez a szám-alapú `LOOP / NEXT` segéd futásidejű megfelelője.

REPEAT / UNTIL

Mint **egy futásidejű hurok teszttel az alján** — a test mindig lefut legalább egyszer, majd a feltétel dönt a leállásról.

Argumentum	Leírás
Register	`A`, `X` vagy `Y`
Operator	`==`, `!=`, `<`, `<=`, `>`, `>=`
Value	Immediate érték HEX vagy DEC formátumban

Szintaxis:

```
.repeat
    JSR getchar
```

```
.until A == #$00
```

Méret: a hurok testétől és az összehasonlítási formától függ.

Használd a `REPEAT / UNTIL`-t, ha azt akarod, hogy a test legalább egyszer lefusson a kilépési ellenőrzés előtt.

MEMCPY / MEMSET

Mint **kis memória-rutinok, amelyekhez folyton nyúlsz** — a `MEMCPY` egy összefüggő blokkot másol, a `MEMSET` egy tartományt tölt fel egy byte-tal.

Makró	Argumentumok
`MEMCPY`	`src`, `dst`, `size`
`MEMSET`	`addr`, `value`, `size`

Szintaxis:

```
.memcpy src=$C000, dst=$D000, size=$0100
.memset addr=$0400, value=#$20, size=$03E8
```

Generált kód: inline másoló/kitöltő hurkok, a kért mérethez választva.

A 256 byte-ig terjedő méretek rövid 8 bites hurkot használnak. A nagyobb méretek automatikusan 16 bites számlálóra váltanak.

> **2.1:** a méretet már a kód elrendezésekor ismerni kell — szám vagy konstans. A később definiált címke (amelynek értékét az elrendezés még nem ismeri) hiba, nem a helyére nem férő kód.

PRINT / PRINT_CHAR / PRINT_HEX / CLEAR_SCREEN / WAIT_KEY / DELAY / SET_BORDER / SET_BG

PRINT

Mint **PETSCII kimenet a sablonkód nélkül** — egy stringet ír ki `CHROUT`-on át ugyanazzal a kis/nagybetű kezeléssel (`lower`), mint a `.petscii`.

Szintaxis:

```
.print "HELLO"
.print "hello", lower
```

PRINT_CHAR

Egyetlen PETSCII byte-ot ír ki numerikus kód alapján, `CHROUT`-on átküldve. Az érték lehet nevesített const vagy címke is, amely assembleáláskor byte-ra oldódik.

Szintaxis:

```
.print_char 65
.print_char $41
.print_char color
```

PRINT_HEX

Egy 8 bites értéket ír ki hexadecimális szövegként a szokásos KERNAL kimeneti útvonalon.

Szintaxis:

```
.print_hex A
```

CLEAR_SCREEN

Rövidítés a szabványos C64 képernyőtörlés vezérlőkódhoz.

Szintaxis:

```
.clear_screen
```

WAIT_KEY

Vár, amíg egy billentyűt le nem nyomnak, így nem kell minden alkalommal kézzel megírni a `GETIN` hurkot.

Szintaxis:

```
.wait_key
```

DELAY

A kért számú frame-et várja egy közös segédrutinon keresztül. Ezt használd rövid szünetekhez és időzítési résekhez, amikor egy teljes egyéni hurok túlzás lenne. A frame-szám lehet nyers szám vagy nevesített const, és a `.wait` csak a `.delay` aliasa.

Szintaxis:

```
.delay 29
.wait 29
.delay frames=FRAMES_1S
```

SET_BORDER / SET_BG

Kényelmi burkolók a VIC-II szín-regiszterekhez. A szín-érték lehet nyers szám vagy nevesített const, amely 0–15-re oldódik.

Szintaxis:

```
.set_border 6
.set_bg 0
.set_border color
.set_bg color
```

Méret: minden segéd egy pici regiszter-írási szekvenciára vagy rövid KERNAL hívásra fejt ki.

IRQ_SETUP

Egy lépésben állít be egy raszter IRQ kezelőt. A makró beírja az IRQ vektort, engedélyezi a raszter IRQ-kat, beállítja a sort, letiltja a gyakori CIA IRQ forrásokat, és `CLI`-vel tér vissza a normál végrehajtáshoz.

Argumentum	Leírás
Handler	IRQ rutin címkéje (pl. `my_irq`)
Raster	Raszter sor hex vagy decimális formában (pl. `$FA`)

Szintaxis:

```
.irq_setup handler=my_irq, raster=$FA
```

Méret: egy kis beállító szekvencia; a pontos hossz a választott raszter sortól függ.

Ezt akkor használd, ha a gyakori „SEI / kezelő beírása / IRQ engedélyezés / CLI" sablonkódot szeretnéd anélkül, hogy szétszórnád a programban.

RAND

Mint **egy pici beépített PRNG** — 8 bites pszeudo-véletlen értéket ad egy kompakt zero-page seedből.

Argumentum	Leírás
Seed	Opcionális zero-page seed byte vagy címke (alap `$FB`)

Szintaxis:

```
.rand
```

Méret: néhány byte, a választott implementációs úttól függően.

A generátor játékmenetre, effekt-variációra és gyors tesztadatra való. Szándékosan kicsi, nem kriptográfiailag kifinomult.

SPRITE_INIT

Egy sorban állít be egy VIC-II sprite-ot — ahelyett, hogy ~6 POKE utasítást írnál BASIC-ben. Beállítja a sprite adat-pointerét, bekapcsolja, opcionálisan engedélyezi a multicolor módot, és beállítja a színét.

Argumentum	Leírás
Sprite #	Sprite szám 0–7
Colour	Színindex 0–15 (C64 paletta)

Argumentum	Leírás
Data page	Sprite adat cím / 64 (pl. `$21`, ha az adat a `$0840`-en van)
Multicolor	A sprite multicolor bitjét kapcsolja (`$D01C`)

Szintaxis:

```
.sprite_init 0, 7, $21
.sprite_init 0, 7, $21, multicolor
.sprite_init 0, 7, $21, mono
```

Generált kód:

```
    LDA #$21
    STA $07F8       ; sprite pointer regiszter ($07F8 + N)
    LDA $D015
    ORA #$01        ; sprite 0 engedély bit beállítása
    STA $D015
    LDA $D01C
    AND #$FE        ; sprite 0 multicolor bit törlése
    STA $D01C
    LDA #$07
    STA $D027       ; sprite 0 szín-regiszter
```

Méret: 26 byte.

> **Sprite data page:** `data_address ÷ 64`. Az alapértelmezett BASIC SYS stubbal a `JMP main` utáni `ALIGN 64` a sprite adatot a `$0840`-re helyezi → page = `$21`.

SPRITE_POS

Mint a `POKE 53248, x : POKE 53249, y` BASIC-ben — beállítja egy sprite kezdőpozícióját. A koordináták assembleáláskor be vannak égetve, és const-okat is elfogadnak; animációhoz használj `INC` / `DEC`-et közvetlenül a sprite regiszteren.

Argumentum	Leírás
Sprite #	Sprite szám 0–7
X	Vízszintes pozíció 0–319
Y	Függőleges pozíció 0–255

Szintaxis:

```
.sprite_pos 0, 152, 100
```

Generált kód (példa: sprite 0, X=152, Y=100):

```
    LDA #$98        ; X alsó byte
    STA $D000       ; sprite 0 X regiszter
    LDA $D010
```

```
    AND #$FE        ; sprite 0 X MSB törlése (X ≤ 255)
    STA $D010
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y regiszter
```

X > 255 esetén a makró a `$D010` megfelelő bitjét beállítja ahelyett, hogy törölné.

Méret: 18 byte.

> **Megjegyzés:** a `SPRITE_POS` beégeti az X/Y-t a kódba (`LDA #$xx`). Sprite futásidejű animálásához használj `INC $D000` / `DEC $D000` -t — lásd a `sprite-macro-demo` mintát.

WAIT_RASTER

Megvárja, míg a VIC-II elektronsugár elér egy adott scanline-t — mint a TV frame-re szinkronizálás. Tedd ezt a game hurkod tetejére a sprite tearing megelőzéséhez. Nincs JSR, nem kell címke.

Argumentum	Leírás
Raster line	Cél raszter sor hexben (pl. `FF` = 255. sor)

Szintaxis:

```
.wait_raster $FF
```

Generált kód:

```
wait:
    LDA $D012       ; aktuális raszter sor
    CMP #$FF        ; cél sor
    BNE wait        ; vissza a hurokba (-7 byte)
```

Méret: 7 byte (a `BNE` offset `$F9` = −7 mindig az `LDA` -ra mutat vissza).

> **Tipp:** tedd a `WAIT_RASTER` -t a game hurkod tetejére a kijelzővel való szinkronizáláshoz és a sprite tearing megelőzéséhez.

JOYSTICK

Mint a **`PEEK($DC00)`** olvasása, majd a sprite pozíció POKE-olása — de egy sorban. Beolvassa az egyik CIA joystick portot, és ennek megfelelően állítja egy sprite X/Y regisztereit. Teljesen inline, nem kell JSR.

Argumentum	Leírás
Port	`1` = port 1 (`$DC01`) vagy `2` = port 2 (`$DC00`)
Sprite #	Sprite szám 0–7 (melyik X/Y regiszterpár frissül)

Szintaxis:

```
.joystick 2, 0
```

Generált kód (port 2, sprite 0):

```
    LDA $DC00       ; CIA port 2 olvasása
    LSR             ; 0. bit → carry (Fel)
    BCS skip_up     ; carry beállítva = NINCS nyomva
    DEC $D001       ; Y–1 (fel mozgatás)
skip_up:
    LSR             ; 1. bit → carry (Le)
    BCS skip_down
    INC $D001       ; Y+1 (le mozgatás)
skip_down:
    LSR             ; 2. bit → carry (Balra)
    BCS skip_left
    DEC $D000       ; X–1 (balra mozgatás)
skip_left:
    LSR             ; 3. bit → carry (Jobbra)
    BCS skip_right
    INC $D000       ; X+1 (jobbra mozgatás)
skip_right:
```

Joystick bittérkép (active-LOW — bit = 0 jelenti a nyomást):

Bit	Irány	CIA regiszter
0	Fel	$DC00 (port 2) / $DC01 (port 1)
1	Le	
2	Balra	
3	Jobbra	
4	Tűz	(ez a makró nem kezeli)

Méret: 27 byte. A `BCS` offset mindig `+3` (átugorja a következő 3 byte-os `DEC` / `INC abs` utasítást).

Változós mód — `.joystick port, vx, vy[, fire]`: sprite mozgatása helyett az irányt változókba írja: `vx` / `vy` = `-1` / `0` / `+1` (`$FF` / `$00` / `$01`), az opcionális `fire` = `1` , amíg a gomb le van nyomva, egyébként `0` . Görgetett kamerához, saját fizikájú játékoshoz vagy menükhöz. A `3` -as port egyszerre olvassa mindkét joystick portot (`LDA $DC00` / `AND $DC01`), így bármelyik port működik.

```
.joystick 2, vx, vy          ; 35 byte
.joystick 3, vx, vy, fire    ; mindkét port + tűz: 45 byte
```

Tipikus használat: tedd egy `gameloop` címke belsejébe, elöl `WAIT_RASTER` -rel:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    JMP gameloop
```

> **2.1:** a portot már a kód elrendezésekor ismerni kell (szám vagy konstans), mert a 3-as port 3 bájttal hosszabb kódot ad; a később definiált címke hiba.
>
> **MOUSE**

Beolvas egy Commodore 1351 arányos egeret, és mozgat egy sprite-ot. Teljesen **inline** — nem kell JSR vagy címke. A makró kiválasztja a CIA portot, megvárja, míg a SID paddle bemenetek beállnak, majd dekódolja a delta mozgást a szabványos 1351 driver mintával, és alkalmazza a sprite regiszterekre.

Argumentum	Leírás
Port	`1` = CIA `$DC00` `7:6` bitek = `%01`; `2` = `%10`
Sprite #	Sprite szám 0–7
ZP byte X	Zero-page cím (hex) az előző POTX minta tárolására (pl. `FD`)
ZP byte Y	Zero-page cím (hex) az előző POTY minta tárolására (pl. `FE`)

Generált kód forma (port 1, sprite 0, ZP `$FD` / `$FE`):

```
    ; CIA port kiválasztás + beállás
    LDA $DC00
    AND #$3F
    ORA #$40
    STA $DC00
    LDX #$67
wait:
    DEX
    BNE wait

    ; X tengely — szabványos 1351-stílusú 7 bites delta dekódolás
    LDA $D419
    TAY
    SEC
    SBC $FD
    AND #$7F
    LDX #$00
    CMP #$40
    BCS xneg
    LSR A
    BEQ xdone
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
    JMP xdone
xneg:
    ORA #$C0
    CMP #$FF
    BEQ xdone
```

```
    SEC
    ROR
    DEX
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
xdone:

    ; Y tengely – ugyanaz a dekódolás, majd invertálás alkalmazás előtt
    LDA $D41A
    TAX
    SEC
    SBC $FE
    AND #$7F
    CMP #$40
    BCS yneg
    LSR A
    BEQ ydone
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
    JMP ydone
yneg:
    ORA #$C0
    CMP #$FF
    BEQ ydone
    SEC
    ROR
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
ydone:
```

Méret: 142 byte.

Szintaxis:

```
.mouse port, spriteNum, zpX, zpY
; példa:
.mouse 2, 0, FD, FE
```

Fontos: az első hívás előtt inicializáld a zero-page byte-okat az aktuális POTX/POTY értékekkel, hogy elkerüld az ugrást az első frame-en:

```
;     port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
;     port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
```

```
    LDA $D419 : LSR A : AND #$3F : STA $FD
    LDA $D41A : LSR A : AND #$3F : STA $FE
```

> **Tipp:** olvasd az egeret frame-enként egyszer — tedd a `WAIT_RASTER`-t a game hurokban a `MOUSE` elé.

SPRITE_COL

Mint a **`PEEK($D01E)`** BASIC-ben — ellenőrzi a VIC-II hardveres ütközés-regisztereket, és megmondja, hogy egy sprite eltalált-e egy másik sprite-ot vagy a hátteret. Teljesen inline, nem kell JSR.

Argumentum	Leírás
Sprite #	Sprite szám 0–7 (melyik sprite bitjét ellenőrizze)
Collision type	`Sprite-Sprite ($D01E)` — ütközés másik sprite-tal; `Sprite-Background ($D01F)` — ütközés a háttér-grafikával

Szintaxis:

```
.sprite_col 0, sprite
.sprite_col 0, background
```

Generált kód (sprite 0, sprite–sprite):

```
    LDA $D01E       ; sprite-sprite ütközés-regiszter olvasása (törli is!)
    AND #$01        ; 0. bit izolálása (sprite 0)
                    ; A ≠ 0 → ütközés történt
```

Méret: 5 byte.

> **Fontos:** a `$D01E` / `$D01F` olvasása **törli a regisztert**. Olvasd frame-enként egyszer, és azonnal reagálj az eredményre `BEQ` / `BNE`-vel.

Tipikus használat:

```
gameloop:
    .wait_raster $FF
    .joystick 2, 0
    .sprite_col 0, sprite
    BEQ no_hit          ; A = 0 → nincs ütközés
    LDA #$02
    STA $D020           ; piros keret = találat!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; világoskék keret = tiszta
    JMP gameloop
```

> **Lásd még:** `collision-demo` minta — zöld labda (sprite #0) vs. piros kereszt (sprite #1).

LOADFILE

Mint a `LOAD "file",8` BASIC-ben — betölt egy fájlt egy D64 lemezről futásidőben a KERNAL LOAD rutinjával. Ezzel tölthetsz adatot, zenét vagy extra kódot lemezről, miközben a programod fut.

Argumentum	Leírás
Filename	Fájlnév a lemezen (max 16 karakter, automatikus nagybetűsítés; a `,`, `"`, `/`, `\`, `:`, `*`, `?`, `<`, `>`, `|` karakterek kiszűrődnek)
Device	Eszközszám 8–30 (alap `8`)
Override address (opcionális)	Hex betöltési cím (pl. `C000`). Ha beállított, a fájl erre a címre töltődik (`sec=0`, a PRG fejlécet figyelmen kívül hagyva). Hagyd üresen a fájl saját 2 byte-os PRG fejlécének használatához (`sec=1`).
Error label (opcionális)	Ha beállított, egy `BCS` utasítás generálódik a JSR LOAD után. Ha a KERNAL carry beállítással (hiba) tér vissza, a végrehajtás erre a címkére ugrik.

Szintaxis:

```
.loadfile "DEMO-COLORS", 8
.loadfile "DEMO-COLORS", 8, $C000
.loadfile "DEMO-COLORS", 8, $C000, error_label
```

Generált kód szerkezete:

```
    JMP skip_filename      ; átugrik az inline fájlnéven
    .byte "DEMO-COLORS"    ; fájlnév byte-ok (PETSCII, 11 karakter)
skip_filename:
    LDA #11                ; fájlnév hossza
    LDX #<fname            ; pointer lo
    LDY #>fname            ; pointer hi
    JSR $FFBD              ; SETNAM
    LDA #1                 ; logikai fájl #1
    LDX #8                 ; device 8
    LDY #0                 ; sec=0 (override cím) vagy #1 (fájl saját címe)
    JSR $FFBA              ; SETLFS
    LDA #0                 ; LOAD parancs (nem VERIFY)
    JSR $FFD5              ; LOAD
    BCS fail               ; (csak ha van error label)
```

Méret: `3 + fájlnév_hossz + 9 (SETNAM) + 9 (SETLFS) + (4 ha override) + 5 (LOAD) + (2 ha error label)` byte. Minimum 27 byte.

> **Fontos:** a fájlnév inline tárolódik a gépi kódban közvetlenül egy `JMP skip_filename` után. A lemezen lévő fájlnévnek nagybetűs PETSCII-nek kell lennie — ami megegyezik a sima ASCII nagybetűkkel (`A` – `Z`). A makró ezt automatikusan kikényszeríti.

> **Éles programokhoz mindig használj error label-t** — ha a fájl nincs meg, a KERNAL beállítja a carry flag-et, és a végrehajtás átcsúszik arra, ami utána van.

> **Lásd még:** `loadfile-demo` minta — `DEMO-COLORS.PRG` betöltése D64-ről BCS hibaágyal és vizuális hibaképernyővel.

EXODECRUNCH

Programon belüli **Exomizer kitömörítés**. Ezt a makrót közvetlenül egy olyan `LOADFILE` után használd, amely egy Exomizer `mem`-módú tömörített streamet töltött be — az EXODECRUNCH visszafelé csomagolja ki a streambe ágyazott címre.

Argumentum	Leírás
Depacker address	Ahol a depacker kód él a memóriában (alap `B000`). 16 bites hex cím kell legyen.

Szintaxis:

```
.exodecrunch
.exodecrunch depacker=$B000
```

Generált kód (19 byte):

```
    LDA $AE          ; KERNAL load-end lo (az előző LOAD állította be)
    STA $04          ; depacker src-end pointer lo
    LDA $AF          ; KERNAL load-end hi
    STA $05          ; depacker src-end pointer hi
    LDA #$36         ; BASIC ROM ki ($A000-$BFFF RAM lesz)
    STA $01
    JSR depacker     ; a depacker hátulról előre olvas a $04/$05-ön át
    LDA #$37         ; alapértelmezett mapping visszaállítása (BASIC + KERNAL + I/O)
    STA $01
```

Hogyan működik:

1. A KERNAL `LOAD` ($FFD5) a ZP `$AE/$AF`-t az utoljára betöltött byte utáni pozícióra állítja. Az EXODECRUNCH ezt a ZP `$04/$05`-re másolja, ami a hivatalos Exomizer visszafelé-forrás-vég konvenció.
2. A depacker jellemzően a `$B000`-en van (a BASIC ROM-ra mappelt tartományon belül). A makró `$01 = $36`-ra kapcsol, hogy a CPU ott RAM-ot lásson a JSR alatt, majd utána visszaállítja `$01 = $37`-re.
3. A kitömörítési célcím **magába a tömörített streambe van kódolva**, amikor `exomizer mem -l <load> file, <target>`-rel tömörítesz — a depacker a stream első byte-jaiból olvassa ki.

Depacker bináris: az előre elkészített visszafelé-depacker az `exo-decrunch.bin` (a C64 Visual Assembler `samples/` mappájában) (477 byte, ORG $B000). Ez a hivatalos `exodecrunch.asm` Kick Assembler-es burkolása, minden olvasáshoz hozzáadott `INC $D020`-szal, hogy a kitömörítés alatt látható keret-villantás legyen. Tedd a programodba `.incbin`-nel a depacker címén.

Nyers bemenet: az extra nyers fájlként, a Dst címre töltve megy az Exomizerhez (`file@<Dst>`), így a depacker minden bájtot, az elsőtől kezdve, pontosan a beírt címre ír.

> **Lásd még:** `exo-multicolor-demo` minta — teljes végponttól végpontig példa: LOADFILE-lal egy tömörített multicolor bitmap a $C000-re, EXODECRUNCH kicsomagolja a $2000-re, majd screen → $0400 és color → $D800 másolás, és a VIC-II multicolor bitmap módra váltás.

REU_CHECK

Detektálja, hogy be van-e dugva egy Commodore RAM Expansion Unit (REU) — mint a `PEEK($D010)` ellenőrzése, hogy jelen van-e a hardver. Két minta beírásával és visszaolvasásával tesztel a REU `$DF04` regiszterén.

Argumentum	Leírás
Nincs	Nincs operandus

Generált kód (34 byte):

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF   ; Z=0 => REU jelen van
BNE done
fail:
LDA #$FF
CMP #$FF   ; Z=1 => nincs REU
done:
```

> A makró normalizálja az eredményt, hogy a következő elágazás egyszerű maradjon: `BNE` = REU jelen van, `BEQ` = REU hiányzik.

Szintaxis:

```
.reu_check
```

Eredmény a flag-ekben:

- **Z = 0** (eredmény ≠ 0) → REU jelen van → használj `BNE`-t
- **Z = 1** (eredmény = 0) → nincs REU → használj `BEQ`-t

Nincs argumentuma — a makró minden alkalommal ugyanazt a kódot generálja.

Tipikus használat:

```
.reu_check
BEQ no_reu        ; kihagyás, ha nincs REU
; ... REU kód itt ...
no_reu:
```

REU_STASH / REU_FETCH / REU_SWAP

DMA blokk-átvitel a C64 RAM és a REU bővítő memória közt — mint egy nagyon gyors POKE hurok, de a CPU nem dolgozik (a REU chip másolja az adatot, míg a CPU meg van állítva). Egy `$1000` byte-os átvitel gyakorlatilag azonnali.

Makró	Irány	`$DF01` parancs
`REU_STASH`	C64 RAM → REU	`$90`
`REU_FETCH`	REU → C64 RAM	`$91`
`REU_SWAP`	C64 RAM ↔ REU	`$92`

Argumentumok:

Argumentum	Leírás	Példa
C64 address	Forrás/cél a C64 RAM-ban (hex)	`C000`
REU address	Forrás/cél a REU-ban (hex, 16 bites)	`0000`
REU bank	REU memória-bank (0–7)	`0`
Length	Átviendő byte-ok száma (hex, 16 bites)	`1000`

Szintaxis:

```
.reu_stash $C000, $0000, 0, $1000
.reu_fetch $C000, $0000, 0, $1000
.reu_swap $C000, $0000, 0, $1000
```

Generált kód (40 byte):

```
LDA #c64lo    STA $DF02    ; C64 cím LO
LDA #c64hi    STA $DF03    ; C64 cím HI
LDA #reuLo    STA $DF04    ; REU cím LO
LDA #reuHi    STA $DF05    ; REU cím HI
LDA #bank     STA $DF06    ; REU bank
LDA #lenLo    STA $DF07    ; hossz LO
LDA #lenHi    STA $DF08    ; hossz HI
LDA #$00      STA $DF09    ; address control (fix)
LDA #$00      STA $DF0A    ; interrupt mask (fix)
LDA #cmd      STA $DF01    ; DMA végrehajtás ($90/$91/$92 = stash/fetch/swap, azonnali)
```

> **Megjegyzés:** a parancsok `$90/$91/$92`-t használnak (4. bit be = azonnali DMA mód). A `$DF01`-be írás elindítja az átvitelt; a CPU folytatódik, amikor befejeződik.

TURBO_SET

Beállítja az **Ultimate-64 (U64) CPU sebességet** a `$D031` regiszteren keresztül. Valódi C64-en vagy más emulátoron nincs hatása.

Argumentumok:

Argumentum	Leírás	Tartomány
Speed	CPU sebesség-index	0 = 1 MHz … 7 ≈ 10 MHz … 15 ≈ 48 MHz
Badline	Badline emuláció	Engedélyezve (C64 kompatibilis) / Letiltva (turbo)

A sebesség-byte így számolódik: `(speedIndex & 0x0F) | (badline_letiltva ? 0x80 : 0x00)`.

Generált kód (5 byte):

```
A9 xx   LDA #speed_byte
8D 31 D0   STA $D031
```

Szintaxis:

```
.turbo_set 7,0    ; speed=7 (~10 MHz), badline engedélyezve
.turbo_set 15,1   ; speed=15 (~48 MHz), badline letiltva
```

> **Megjegyzés:** ez a makró csak U64 hardvert érint. Valódi C64-en vagy más emulátoron a `$D031`-be ír, ami hatással lehet a CIA-ra, vagy figyelmen kívül maradhat.

U64_PALETTE / U64_PALETTE_COLOR / U64_PALETTE_RESET

Az **Ultimate 64 / C64 Ultimate videó-palettáját** állítják az Ultimate Command Interface-en át (`$DF1C` – `$DF1F`, 3.15-ös vagy újabb firmware, a menüben bekapcsolt Command Interface). Új a 2.2-ben.

Makró	Szintaxis	Méret	Parancs (cél `$04`)
U64_PALETTE	`.u64_palette "fájl.vpl"`	109 byte	`$52` + 16 × R, G, B
U64_PALETTE_COLOR	`.u64_palette_color index, $RRGGBB`	65 byte	`$53` index, R, G, B
U64_PALETTE_RESET	`.u64_palette_reset`	61 byte	`$54` (a menü palettája)

A `.u64_palette` fájlját úgy keresi, mint egy `.incbin` fájlt. Lehet VICE `.vpl` (soronként `RR GG BB dither`, `#` megjegyzés), GIMP `.gpl`, bármilyen szöveg legalább 16 `#RRGGBB` / `$RRGGBB` / `0xRRGGBB` értékkel, vagy Photoshop `.act` (bináris, legalább 768 byte); az első 16 szín számít. A színek a generált kódba kerülnek, máshová nem kerül semmi a memóriában. A `.u64_palette_color` indexe és színe kifejezés: az index 0–15, a szín `$000000` – `$FFFFFF`.

Generált kód (59 byte, utána a parancs bájtjai; az A és X elvész):

```
        LDA $DF1D       ; azonosító: $C9, ha az interfész be van kapcsolva
        CMP #$C9
        BNE done        ; nincs interfész (VICE, régebbi firmware): kihagyja
idle:   LDA $DF1C
        AND #$30        ; állapot
        BEQ ready
        AND #$20
        BEQ idle        ; dolgozik
        LDA #$02        ; korábbi parancs adata: elfogadja
        STA $DF1C
        BNE idle
ready:  LDX #$00
send:   LDA data,X
        STA $DF1D
        INX
        CPX #len
        BNE send
        LDA #$01        ; parancs indítása
```

```
        STA $DF1C
busy:   LDA $DF1C
        AND #$30
        CMP #$10
        BEQ busy
        LDA #$02        ; a válasz elfogadása
        STA $DF1C
        BNE done
data:   .byte $04, parancs, ...
done:
```

Hibák: `u64PaletteNeedsFile` (nincs fájl), `u64PaletteBadFile` (nem 16 szín), `u64PaletteColorSyntax`, `u64PaletteColorRange`.

SUPERCPU_DETECT

Ellenőrzi, hogy telepítve van-e egy **CMD SuperCPU** gyorsító — mint a `PEEK($D0B8)`, hogy `$FF`-től eltérő értéket ad-e vissza.

Generált kód (5 byte):

```
AD B8 D0   LDA $D0B8
C9 FF      CMP #$FF
```

Eredmény a flag-ekben:

- **Z = 0** → SuperCPU jelen van → használj `BNE`-t
- **Z = 1** → SuperCPU nincs → használj `BEQ`-t

Nincs argumentuma.

Szintaxis:

```
.supercpu_detect
```

Tipikus használat:

```
.supercpu_detect
BEQ no_scpu       ; kihagyás, ha nincs SuperCPU
; ... SuperCPU turbo kód itt ...
no_scpu:
```

TURBO_ENABLE

Be- vagy kikapcsolja a **CMD SuperCPU turbo módot**. Előbb hívj `SUPERCPU_DETECT`-et, és hagyd ki ezt, ha a SuperCPU nincs jelen.

Mód	Regiszter	Hatás
Enable	`$D07A`	Turbo bekapcsolása (SuperCPU-val akár 20 MHz)

Mód	Regiszter	Hatás
Disable	`$D07B`	Vissza az 1 MHz-es kompatibilitási módba

Generált kód (5 byte):

```
A9 00         LDA #$00
8D 7A D0      STA $D07A    ; (vagy $D07B a letiltáshoz)
```

Szintaxis:

```
.turbo_enable on
.turbo_enable off
```

> **Megjegyzés:** előbb hívj `SUPERCPU_DETECT`-et, és ágazz el a makró körül, ha a SuperCPU nincs jelen.

MAP_COPY

Egy tilemap-et másol egy forráscímről a screen RAM-ba (és opcionálisan a color RAM-ba) `LDA abs,X` / `STA abs,X` hurkok sorozatával. Hurok-iterációnként egy 256 byte-os lap másolódik; a végén egy részleges lap `CPX #rem / BNE`-vel áll le. Nem kell JSR — minden kód inline generálódik.

Argumentum	Leírás
Source addr (screen)	Hex cím, ahol a map adat betöltés után él (pl. `C000`)
Screen RAM dest	Hová másolja a screen code-okat (pl. `0400`)
Size (bytes)	Összes másolandó byte — jellemzően `$03E8` = 1000 (40×25 karakter)
Color forrás	`auto`: a color adat közvetlenül a screen code-ok után van (`source + size`); vagy egy konkrét color forráscím. A color adat egy második menetben a color RAM célcímre másolódik. Elhagyva (a color célcímmel együtt) csak a screen code-ok másolódnak
Color RAM dest	A color adat célja — alap `D800` (C64 color RAM)

Generált kód (1000 byte-os map, csak screen):

```
    LDX #$00
   LDA $C000,X   ; 0. lap
   STA $0400,X
   INX
   BNE *-9       ; hurok, míg X vissza nem fordul 0-ra (256 iteráció)
   LDA $C100,X   ; 1. lap
   STA $0500,X
   INX
   BNE *-9
   LDA $C200,X   ; 2. lap
   STA $0600,X
   INX
   BNE *-9
   LDA $C300,X   ; maradék – 232 byte
```

```
    STA $0700,X
    INX
    CPX #$E8
    BNE *-11
```

Méret: `2 (LDX) + fullPages×9 + (rem > 0 ? 11 : 0)` byte szakaszonként. A combined mód megduplázza (screen szakasz + azonos color szakasz).

Párosítás a Map Editorral:

A C64 Visual Assembler Map Editora (**Save map + color RAM**) egyetlen binárist exportál, ahol az első `size` byte screen code, a következő `size` byte pedig color RAM érték. Használd a `.map_copy`-t `auto` color forrással, és a forráscímet oda mutasd, ahova ez a fájl betöltődik (pl. `.incbin`-nel a `$C000`-en):

```
    .incbin "map-color.bin", $C000   ; screen code-ok $C000–$C3E7, color $C3E8–$C7CF
    ; ...
    .map_copy $C000, $0400, 1000, auto, $D800
```

Szintaxis:

```
.map_copy $C000, $0400, 1000               ; csak screen
.map_copy $C000, $0400, 1000, auto, $D800  ; combined (color a src+size címen)
.map_copy $C000, $0400, 1000, $C3E8, $D800 ; explicit color forráscím
```

MAP_COPY16X16

Egy 16×16 karakteres területet másol egy kompakt 256 byte-os screen-code blokkból plusz egy hozzá tartozó 256 byte-os Color RAM blokkból. Charset Canvas exportokhoz és kis tile/kép darabokhoz való, ahol tizenhat különálló MAP_COPY sor zajos lenne.

Alapértelmezett elrendezés:

Adat	Alapértelmezett cím
16×16 screen code-ok	Forráscím (`src`)
16×16 color értékek	`src + 256`
Screen RAM cél	`$0400 + row×40 + col`
Color RAM cél	`$D800 + row×40 + col`

Szintaxis:

```
.map_copy16x16 $3000, 12, 4
.map_copy16x16 $3000, 12, 4, $0400, $3100, $D800
```

A rövid forma a screen byte-okat a `$3000`-ről, a color byte-okat a `$3100`-ról másolja, és a 16×16 blokkot a 12. oszlop, 4. sorba helyezi. Az érvényes bal-felső pozíciók `col = 0..24` és `row = 0..9`, így a teljes 16×16 terület a 40×25-ös C64 szövegképernyőn marad.

Generált viselkedés:

- Tizenhat inline sor-másolást generál.
- Minden sor 16 screen byte-ot és 16 color byte-ot másol.
- Nem kell JSR; a kód közvetlenül a makró pozíciójára kerül.
- Normál karakter móddal és multicolor karakter móddal is működik; a Color RAM byte-ok hordozzák minden cella karakterszínét / multicolor engedély bitjét.

Tipikus párosítás Charset Canvas-szal:

```
* = $0801
    SEI
    LDA #$00
    STA $D021
    LDA #$05
    STA $D022
    LDA #$0D
    STA $D023
    .map_copy16x16 $3000, 12, 4
loop:
    JMP loop

.incbin "16x16+ram+color.bin", $3000
.incbin "16x16+charset.bin", $2000
```

SPRITE_ANIM

Minden híváskor lépteti egy sprite animációs frame-jét, és frissíti a VIC-II sprite adat-pointerét. Tárolj frame-enként egy byte-ot egy táblában (a sprite adat-lap száma = `data_address / 64`), mutass rá a SPRITE_ANIM-mal, és hívd game frame-enként egyszer — nem kell JSR.

Argumentum	Leírás
Sprite #	Sprite szám 0–7
Frame list address	A frame-tábla hex címe — frame-enként egy byte, minden byte = sprite lap (`data_addr / 64`)
Frame count	Frame-ek száma összesen (1–255)
Frame ZP	Frame-számlálóként használt zero-page byte (pl. `FB`)

Generált kód (sprite 0, 4 frame, ZP `$FB` , lista a `$C100` -on):

```
    INC $FB         ; frame-számláló léptetése
    LDA $FB
    CMP #$04        ; frame-szám
    BCC *+6         ; ha számláló < szám, ugorjuk át a resetet
    LDA #$00
    STA $FB
    TAX             ; X = aktuális frame-index
    LDA $C100,X     ; ehhez a frame-hez tartozó sprite adat-lap betöltése
    STA $07F8       ; sprite 0 pointer frissítése ($07F8 + sprite#)
```

Méret: 19 byte.

Tipikus használat:

```
frameTable:
    .byte $21, $22, $23, $24   ; 4 frame a $0840, $0880, $08C0, $0900 címeken

gameloop:
    .wait_raster $FF
    .sprite_anim 0, $C100, 4, $FB
    JMP gameloop
```

Szintaxis:

```
.sprite_anim spriteNum, frameListAddr, frameCount, zpByte
; példa:
.sprite_anim 0, C100, 4, FB
```

> **Tipp:** tedd a frame-táblát `.rawbytes`-szal fix címre. A számláló ZP byte-ot (`$FB`) `$00`-ra kell inicializálni az első hívás előtt. Ha a kódod máshoz használja a `$FB`-t, válassz szabad ZP helyet.

SCORE_BCD

Egy fix pontszámot ad hozzá egy memóriában tárolt több-byte-os BCD pontszámhoz, majd minden számjegyet screen RAM-ba renderel screen-code karakterként. A 6502 decimal módot (`SED` / `CLD`) használja carry-biztos BCD aritmetikához — nem kell kézzel carryzni.

Argumentum	Leírás
Score address	A BCD pontszám-byte-ok hex címe (pl. `C200`). Alsó byte először.
Digits	BCD byte-ok száma (minden byte két számjegyet tárol: `$99` = „99"). `4` byte = akár 99999999.
Add points	Híváskénti decimális hozzáadandó érték (pl. `100`).
Screen address	Hová írja a számjegy screen code-okat (pl. `0400`). Számjegyenként egy byte (felső nibble először).

Szintaxis:

```
.score_bcd $C200, 4, 100, $0400
```

Generált kód (4 byte, +100 pont, pontszám a `$C200`-on, screen a `$0400`-on):

```
    SED             ; BCD / decimal mód engedélyezése
    CLC
    LDA $C200       ; 0. byte (1–2. számjegy)
    ADC #$00        ; a 100 alsó byte-ja BCD-ben = $00
    STA $C200
    LDA $C201       ; 1. byte (3–4. számjegy)
    ADC #$01        ; a 100 középső byte-ja BCD-ben = $01 (carry propagál)
    STA $C201
    LDA $C202
    ADC #$00
    STA $C202
    LDA $C203
    ADC #$00
    STA $C203
```

```
    CLD             ; vissza bináris módba

    ; számjegyek renderelése
    LDX #$00
digit_loop:
    LDA $C200,X
    PHA
    LSR : LSR : LSR : LSR  ; felső nibble → alsó nibble
    ORA #$30         ; + '0' screen code
    STA $0406,X      ; jobbra igazítva: screen_addr + (digits*2 - 2) - X*2
    PLA
    AND #$0F         ; alsó nibble
    ORA #$30
    STA $0407,X
    INX
    CPX #$04
    BNE digit_loop
```

Méret: `3 + digits×8` byte (SED + CLC + CLD overhead + BCD byte-onként 8 byte az ADC-hez + display hurok).

Szintaxis:

```
.score_bcd $C200, 4, 100, $0400
```

> **Tipp:** inicializáld a pontszám-byte-okat `$00`-ra induláskor. A pontszám-cím zero page-ben vagy abszolút RAM-ban legyen — ne ROM-ban. A screen-cím a legbaloldalibb számjegycellára mutasson; a számjegyek balról jobbra íródnak (legnagyobb helyi értékű byte először).

> **BCD tartomány:** `digits=4` byte → 8 decimális számjegy → max pontszám 99 999 999. Minden byte két BCD számjegyet kódol: `$00` – `$99`.

---

7. KickAssembler-stílusú szintaxis

A VMA a gyakori KickAssembler-formákat is olvassa, így egy KickAssembler forrás gyakran változatlanul lefordul:

- A `.pc = $1000` és a `*=$1000 "név"` a címet állítja (a szegmensnév nem számít).
- A `.const NÉV = érték` és a `.label NÉV = érték` konstanst ad; a `.label NÉV = *`, `.label NÉV = * + 1` címkét tesz az aktuális címre (vagy mellé).
- `.macro név(p1, p2) { … }` — kapcsos zárójeles makrótörzs a `.macro … .endm` mellett; a `:név(args)` és a `név(args)` hívja.
- `név: { … }` scope: a benne lévő címkék a `név` lokálisai (kívülről `név.címke`).
- A `@címke` címkék megtartják a `@`-ot.
- A képernyő-koordináta nélküli `.text "…"` a kódolt bájtokat tárolja, mint a `.byte`. Több elem `,`-vel vagy `+`-szal fűzhető: `"…"`, `@"…"` a `\n` `\r` `\t` `\$NN` escape-ekkel, és `$NN` / decimális bájtok. Az `.encoding "név"` a következő `.text` sorok kódolását választja: `screencode_mixed` (alap), `screencode_upper`, `petscii_mixed`, `petscii_upper`, `ascii`; a kódolásban nem létező karakter hiba.
- A `.abs` / `.a` és `.zp` / `.z` mnemonik-utótag abszolút vagy zero-page címzést kényszerít: `sta.abs $00FC,x`, `ldx.zp ptr,y`.
- `// …` megjegyzés.

A KickAssembler nyelv többi részét (`.for` , `.var` / `.eval` , `.function` , `.pseudocommand` , `#import` , `#if` , `LoadBinary` , szegmensek …) a C64 Visual Assembler Kick-előfeldolgozója kezeli, amely egy KickAssembler forrásfát VMA szöveggé alakít.

8. Parancssor

Új a 2.1.0-ban: `--crt` / `--crt-name` (cartridge) és `--d64` / `--add` / `--disk-name` (lemezkép).

```
vma build <input.asm> [kapcsolók]
vma --help
vma --version
```

Kapcsoló	Jelentés
`-o FÁJL`	A kiírandó PRG (alapból a bemenet `.prg` kiterjesztéssel)
`--base-dir MAPPA`	Itt keresi először az `.incbin` / `.koala` / `.level` / `.u64_palette` / `.sid` / `.include` fájlokat (alapból a bemenet mappája)
`--search MAPPA`	További keresési mappa a base dir után (ismételhető, sorrendben); a más gépen írt útvonal a puszta fájlnévre esik vissza
`--no-basic-stub`	Nincs `10 SYS…` BASIC sor a kód előtt
`--raw`	Csak a memóriakép az első ORG-on: nincs stub, nincs `$C000` -ra tolás
`--crt FÁJL`	Magic Desk cartridge (CRT 19-es típus, 8 × 8K bank, legfeljebb 65 408 bájt): egy loader a betöltési címére másolja a programot, és a SYS címen indítja — a Visual Assembler Build CRT-je. `-o` nélkül nem ír PRG-t
`--crt-name NÉV`	A cartridge neve a CRT fejlécében (alapból `VA CART`)
`--d64 [FÁJL]`	1541-es lemezkép a PRG-vel, a PRG nevén (alapból a PRG `.d64` kiterjesztéssel); c1541 nem kell. `-o` nélkül nem ír PRG-t
`--add FÁJL`	További fájl a lemezre változatlanul (PRG a betöltési címével), a fájl nevén (ismételhető, sorrendben)
`--disk-name NÉV`	A lemez neve (alapból a D64 fájl neve; ID `01`)
`--sym FÁJL`	KickAssembler címkefájl (`.label NÉV=$xxxx`)
`--vs FÁJL`	VICE monitor címkefájl (`al C:xxxx .NÉV` ; `x64sc -moncommands FÁJL`)
`--dbg FÁJL`	RetroDebugger / C64 Debugger forrástérkép (XML)
`--debug`	Mindhárom a PRG mellé (`out.sym` , `out.vs` , `out.dbg`)
`--json FÁJL`	A build JSON-ként: PRG, SYS cím, szimbólumok, elrendezett sorok, kiírt szegmensek, memórianézet — vagy minden hiba a sorával
`--source-path NÉV`	A `.dbg` -ben megnevezett fő forrás (alapból a bemenet)
`--line-map FÁJL`	JSON tömb: bemeneti sor → a `.dbg` -ben megnevezett sor

Kilépési kód: `0` lefordult, `1` fordítási hiba (`FÁJL:SOR: üzenet` a standard hibakimeneten, soronként egy hiba), `2` hibás használat.

```
vma build game.asm -o game.prg --debug
vma build game.asm --no-basic-stub --search ../assets
vma build game.asm --crt game.crt                    # csak a cartridge
vma build game.asm --d64 --add music.prg             # game.d64 a GAME és MUSIC fájlokkal
```

9. Ismert korlátok

A VMA bájtra azonosan hozza a C64 Visual Assembler fordítóját, az egyenetlenségeivel együtt:

- `lda.zp c` → `LDA $0C`: a `.zp` ág a hexa-jegyekből álló szimbólumnevet számnak veszi.
- A paraméter nélküli makró törzse a definíció helyén is lefordul; az első `* =` előtti definíció a program elejére kerül.
- A `.text X, Y, "s", 80` elemzi a karakter-eltolást, de nem használja.
- A `.chardef` bájtjai hexák (`129` = `$129`, hiba).
- A futásidejű `.if` / `.while` / `.until` nem immediate operandusa címként olvasódik: a `%bin` hiba, a `10` hexa (`$0010`).
- A DELAY segédrutin a layout utolsó sora és minden adatszakasz után kerül — kihagyott (`.if 0`) DELAY esetén is.

A 2.1-ben javítva (korábban rossz kódot adtak vagy hibára futottak): a zero page `.const` / `.var` operandus, a `JMP (címke)`, az `lda $1234+1`, a makróban hívott makró, a feloldatlan TURBO_SET / SPRITE_ANIM érték (most hiba), a címkék a `.data` / `.rawbytes` bájtjaiban, a címkére mutató `:név` címke, a JOYSTICK / MEMCPY / MEMSET mérete később definiált címkéből (most hiba), a `.data` / `.rawbytes` automatikus puffere, és az include-ok (a hiányzó hiba, a beemelt fájl saját `.include` / `.incbin` / `.sid` sorai betöltődnek).